

MAC OS TERMINAL HANDBOOK

두 글자로 말하는 사람들 ■

맥 터미널 명령어 사전

맥 터미널, 50개의 단어로 컴퓨터와 대화하는 법

목차

아래 숫자는 각 장이 시작하는 페이지다. PDF 뷰어의 책갈피(bookmark) 패널을 열면 명령어별로도 바로 이동할 수 있다.

1부	두 글자 사전	6
	터미널이 처음이라면	7
01	이동과 둘러보기	14
	pwd	14
	cd	16
	ls	19
	빠른 참조	22
02	파일과 디렉토리 다루기	23
	mkdir	23
	rmdir	26
	touch	28
	rm	30
	cp	33
	mv	36
	ln	38
	빠른 참조	40
03	내용 들여다보기	41
	cat	41
	less	43
	head	45

	tail	47
	echo	49
	빠른 참조	51
04	찾고 뒤지기	52
	find	52
	grep	55
	which	58
	빠른 참조	60
05	텍스트 가공하기	61
	wc	61
	sort	63
	sed	65
	xargs	68
	빠른 참조	70
06	권한과 관리자	71
	chmod	71
	chown	74
	sudo	76
	빠른 참조	78
07	압축과 보관	79
	tar	79
	zip	82
	빠른 참조	84
08	네트워크와 원격 접속	85

ping	85
curl	87
wget	90
ssh	92
scp	94
rsync	96
빠른 참조	99
09 프로세스 관리	100
ps	100
top	102
kill	104
lsof	107
빠른 참조	109
10 비교와 용량 확인	110
diff	110
du	113
df	115
빠른 참조	117
11 셸 환경과 도움말	118
alias	118
export	120
source	122
history	124
man	126
빠른 참조	128

12 맥에서만 되는 것들	129
open	129
pbcopy	132
mdfind	134
caffeinate	136
say	138
빠른 참조	140

2부 두 글자로 문제 풀기	141
-----------------------	-----

13 따라 하기: 프로젝트 하나를 처음부터 끝까지	142
STEP 1 · 작업할 자리 만들기	144
STEP 2 · 도구가 준비됐는지 확인하기	146
STEP 3 · 뼈대 세우기	148
STEP 4 · 설정 파일 채우기	151
STEP 5 · 개발 서버 켜기, 그리고 첫 번째 막힘	153
STEP 6 · 작업하면서 수시로	155
STEP 7 · 올리기 전 점검	158
STEP 8 · 저장하고 내보내기	161
STEP 9 · 내일을 위한 정리	165
오늘 쓴 명령어 정리	167

14 문제 해결 사례집: 증상에서 해결까지	169
CASE 1 · 디스크 공간이 부족하다는 경고가 뜬다	170
CASE 2 · 맥이 갑자기 느려지고 팬이 요란하게 돈다	175
CASE 3 · 외장하드가 "사용 중"이라며 추출되지 않는다	180
CASE 4 · 스크립트를 실행하면 Permission denied가 뜬다	184
CASE 5 · 분명 설치했는데 command not found가 뜬다	188

CASE 6 · 어제 받은 파일이 어디 있는지 모르겠다	192
CASE 7 · 인터넷이 안 된다. 어디가 문제일까	195
CASE 8 · 문서 수십 개에 같은 오타가 있다	199
찾아보기 · 하고 싶은 일로 찾기	204

1부

두 글자 사전

터미널을 처음 여는 사람을 위한 기초에서 시작해, 맥에서 가장 많이 쓰는 명령어 50개를 12개 장으로 나눠 담았다. 명령어마다 쓰는 형식과 실제 예시, 이름의 유래, 옵션 글자의 원래 영단어, 상황별 예제를 사전처럼 찾아볼 수 있다.

-
- 터미널이 처음이라면 · 왜 배워야 할까, 그리고 기초 질문 16가지
 - 01-12장 · 명령어 50개, 장마다 빠른 참조표

컴퓨터는 추측하지 않는다.

당신이 정확히 말해주기를 기다릴 뿐이다.

터미널이 처음이라면

터미널을 익히면 무엇이 달라질까

터미널은 개발자만의 도구가 아니다. 맥을 매일 쓰는 사람이라면 누구나, 몇 개의 단어만 익혀도 같은 컴퓨터를 전혀 다른 수준으로 다루게 된다.

1

반복을 한 줄로

사진 1,000장의 이름 바꾸기, 수백 개 파일 정리처럼 클릭으로 몇 시간 걸릴 일이 명령 한 줄로 끝난다. 한 번 쓴 명령은 저장해 두고 내일도 그대로 다시 쓴다.

2

보이지 않던 것이 보인다

숨김 파일, 폴더의 실제 용량, 지금 몰래 돌고 있는 프로그램. Finder가 감춰 둔 맥의 진짜 상태를 직접 확인할 수 있다.

3

막혔을 때 스스로 푼다

"디스크가 가득 찼다", "포트가 사용 중이다", "Permission denied".
원인을 추측하는 대신 명령으로 확인하고, 재부팅 없이 해결한다.

4

개발 도구의 공용어

Git, Node.js, Python, 클라우드 서버까지. 설치 안내서와 튜토리얼은 거의 모두 터미널 명령어로 쓰여 있다. 터미널을 알면 설명서를 그대로 따라 할 수 있다.

5

먼 컴퓨터도 내 것처럼

화면이 없는 서버, 멀리 있는 컴퓨터도 ssh 한 줄이면 내 맥처럼 다룬다. 웹사이트를 올리고 관리하는 일의 출발점이다.

6

한 번 배우면 오래 간다

50년 전에 태어난 ls, cp, mv가 지금도 똑같이 쓰인다. 앱 메뉴는 업데이트마다 바뀌지만 명령어는 거의 변하지 않고, 맥에서 익힌 명령은 리눅스 서버에서도 그대로 통한다.

같은 일, 두 가지 방법

하고 싶은 일	Finder로 하면	터미널로 하면
사진 500장 중 2024년 것만 다른 폴더로	하나씩 확인하며 끌어다 놓기, 수십 분	<code>mv *2024*.jpg 2024/</code> p.36
맥에서 1GB 넘는 파일 모두 찾기	폴더를 하나씩 열어 크기 확인	<code>find ~ -size +1G</code> p.52
문서 40개의 같은 오타 고치 기	40번 열고, 찾고, 고치고, 저장하기	<code>sed</code> 한 줄 p.199
맥이 느려진 범인 찾기	활동 모니터를 열어 목록 훑어보기	<code>ps aux -r head</code> p.100
만든 웹사이트를 서버에 올리 기	별도 전송 프로그램 설치 와 설정	<code>rsync -avz dist/</code> 서버:경 로 p.96

이제 명령어를 외우기 전에, 처음 터미널을 연 사람이라면 누구나 한 번쯤 품는 질문부터 풀어본다.

Q 터미널이 뭔가요?

평소 쓰는 macOS는 아이콘을 마우스로 눌러 프로그램을 실행하는 방식이다. 터미널(Terminal)은 그 대신 키보드로 글자 명령을 입력해서 컴퓨터에게 직접 지시하는 창이다. Finder가 그림으로 대화하는 창이라면 터미널은 글자로 대화하는 채팅창인 셈이다.

Q 마우스가 있는데 왜 굳이 글자를 쳐야 하나요?

클릭은 한 번에 한 가지 일만 한다. 글자 명령은 반복하고, 조합하고, 저장할 수 있다. 사진 1,000장의 이름을 바꾸는 일은 마우스로는 1,000번의 클릭이지만 터미널에서는 한 줄이다. 한 번 쓴 명령은 파일로 저장해 내일도, 다른 컴퓨터에서도 똑같이 다시 실행할 수 있다. 개발자들이 검은 창을 떠나지 않는 이유다.

Q AI가 명령어를 알려 주는 시대에도 배워야 하나요?

오히려 더 필요하다. AI 챗봇과 코딩 도우미는 문제의 해결책을 터미널 명령으로 내놓는 경우가 많다. 그 명령이 무엇을 하는지 읽지 못하면 그대로 붙여넣는 수밖에 없다. `rm -rf`가 섞인 명령을 뜻도 모른 채 실행하는 것은 운전석에 앉아 눈을 감는 것과 같다. 명령을 읽을 줄 알면 AI의 제안을 검토하고, 고치고, 필요하면 거절할 수 있다. 이 책이 외우기보다 읽고 판단하는 힘을 목표로 하는 이유다.

Q 터미널은 어떻게 여나요?

- 1 Spotlight — `Cmd` + `Space` 누르고 "터미널" 입력 후 Enter
- 2 Finder — 응용 프로그램 → 유틸리티 → 터미널
- 3 Launchpad — Launchpad를 열고 "터미널" 검색

Q 창에 보이는 글자들은 다 뭔가요?

터미널을 열면 가장 먼저 이런 한 줄이 기다리고 있다.

me@MacBook-Pro ~ %

- | | |
|------------------|----------------------|
| 1 me 로그인한 사용자 이름 | 2 MacBook-Pro 컴퓨터 이름 |
| 3 ~ 지금 있는 폴더 (홈) | 4 % 입력을 기다리는 신호 |

이 한 줄 전체를 **프롬프트(prompt)**라 부른다. "무엇을 할까요?"라고 묻는 컴퓨터의 말이다. 그 뒤에 깜빡이는 네모가 **커서**, 지금 타이핑하면 글자가 나타날 자리다. 창 왼쪽 위 세 점은 macOS 공통 버튼으로 빨강은 닫기, 노랑은 최소화, 초록은 전체화면이다.

Q % 나 \$ 같은 기호는 뭔가요?

% 입력 대기 — **zsh**를 쓸 때 (macOS 기본, 2019년 이후)

\$ 입력 대기 — **bash·sh**를 쓸 때 (이전 macOS, 리눅스 다수)

입력 대기, 단 **관리자(root)** 권한으로 접속 중이라는 경고

기호가 다른 건 기능 차이가 아니라 **어떤 셸을 쓰느냐의 전통 차이**다. 인터넷에서 \$ ls처럼 적힌 예시를 보면 맨 앞 기호는 빼고 뒤만 입력하면 된다. 다만 #은 예외다. 지금 무엇이든 바꿀 수 있는 관리자 권한이라는 뜻이니 한 글자 한 글자 조심해야 한다.

Q 셸(shell), 그리고 zsh는 뭔가요?

셸은 입력한 명령을 알아듣고 macOS의 핵심부(커널)에 전달한 뒤 결과를 다시 보여주는 "통역사" 프로그램이다. 이름도 운영체제의 알맹이(커널)를 감싼 조개껍데기(shell)라는 비유에서 왔다.

zsh Z Shell. macOS Catalina(2019)부터 기본 셸. 자동완성과 꾸미기 기능이 풍부함

bash Bourne Again Shell. 2019년 이전 macOS와 대부분 리눅스의 기본 셸

내가 쓰는 셸이 궁금하면 `echo $SHELL`을 입력해보자. 이 책의 예시는 모두 zsh(%) 기준이다.

Q 터미널과 셸은 같은 건가요?

다르다. **터미널**은 창(화면과 키보드)이고, **셸**은 그 안에서 말을 알아듣는 프로그램이다. 전화기와 통화 상대의 관계와 비슷하다. "터미널"은 원래 1970년대 거대한 컴퓨터에 전선으로 연결해 쓰던 키보드 달린 단말기(**terminal**) 기계의 이름이었다. 지금의 터미널 앱은 그 기계를 화면 속에 흉내 낸 것이다.

Q 명령어는 왜 이렇게 짧고 암호 같나요?

유닉스가 태어난 1970년대에는 모니터 대신 종이에 글자를 찍어내는 텔레타이프를 컴퓨터를 썼다. 자판은 무겁고 출력은 느렸기 때문에 한 글자라도 줄이는 게 미덕이었다. 그래서 list는 ls, copy는 cp, move는 mv가 되었다. 50년 전 손가락의 수고가 오늘의 명령어 이름에 그대로 남아 있다.

Q ~ 기호는 왜 "홈 폴더"라는 뜻이 됐나요?

1970년대 유닉스 개발자들이 즐겨 쓰던 ADM-3A라는 단말기에는 HOME 키와 ~ 기호가 같은 키에 새겨져 있었다. 이 우연 덕에 ~가 "집(홈)"을 가리키는 약속이 되었고, 지금도 cd ~는 언제든지 내 홈 폴더로 돌아가는 명령이다.

Q 잘못 입력하면 컴퓨터가 망가지나요?

대부분은 괜찮다. 오타를 내면 zsh: command not found라는 말만 돌아온다. 조심해야 할 것은 되돌릴 수 없는 명령(rm 같은 삭제)과, 관리자 권한으로 실행하는 sudo가 붙은 명령 정도다. 모르는 명령을 인터넷에서 복사해 붙여넣기 전에 한 번만 무슨 뜻인지 확인하는 습관이면 충분하다.

Q sudo는 뭐고, 비밀번호를 쳐도 왜 아무것도 안 보이나요?

sudo는 "superuser do", 이 명령 하나만 관리자 권한으로 실행해 달라는 뜻이다. 그래서 맥 로그인 비밀번호를 묻는다. 이때 글자가 전혀 안 보이는 건 고장이 아니라 옆 사람이 비밀번호의 길이조차 알 수 없게 하려는 설계다. 보이지 않아도 입력은 되고 있으니 그대로 치고 Enter를 누르면 된다.

Q 명령이 멈추지 않아요. 어떻게 빠져나오나요?

Control + **C** 를 누르면 실행 중인 명령을 중단한다. 긴 설명서 화면처럼 스크롤되는 곳에서는 **q** (quit)로 나온다. 이 둘만 알면 터미널에서 길을 잃을 일은 거의 없다.

Q 매번 긴 명령을 다시 쳐야 하나요?

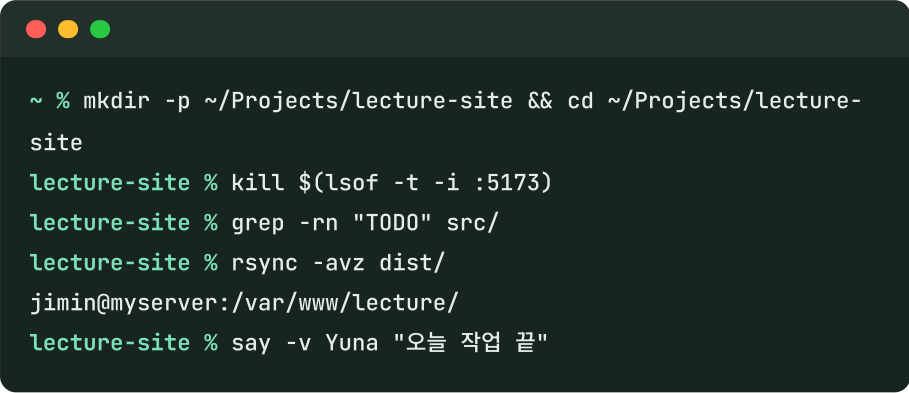
Tab	파일·폴더 이름 앞 몇 글자만 치고 누르면 나머지를 자동완성
↑ / ↓	이전에 입력한 명령을 다시 불러오기
Cmd + K	화면 깨끗이 지우기 (clear 명령과 같음)
history	지금까지 입력한 명령 목록 보기

Q 모르는 명령어는 어디서 배우나요?

터미널 안에 설명서가 들어 있다. `man ls`처럼 `man(manual)` 뒤에 명령어를 적으면 그 명령의 공식 설명서가 열린다. 1971년 첫 유닉스 설명서부터 이어진 전통이다. 다 읽었으면 **q** 로 나오면 된다.

Q 이 책을 다 읽으면 무엇을 할 수 있나요?

예를 들어 이런 하루를 보낼 수 있다. 빈 폴더를 만들고, 막힌 포트를 뚫고, 완성한 사이트를 서버에 올리는 일까지 모두 터미널 몇 줄로 한다.



```
~ % mkdir -p ~/Projects/lecture-site && cd ~/Projects/lecture-site
lecture-site % kill $(lsof -t -i :5173)
lecture-site % grep -rn "TODO" src/
lecture-site % rsync -avz dist/
jimin@myserver:/var/www/lecture/
lecture-site % say -v Yuna "오늘 작업 끝"
```

지금은 암호처럼 보여도 괜찮다. 2부의 13장(p.142)에서 이 하루를 한 줄씩 끝까지 따라가 본다.

터미널에서 길을 잃지 않는 세 가지 기본기. 지금 어디 있는지, 어디로 갈지, 무엇이 있는지.

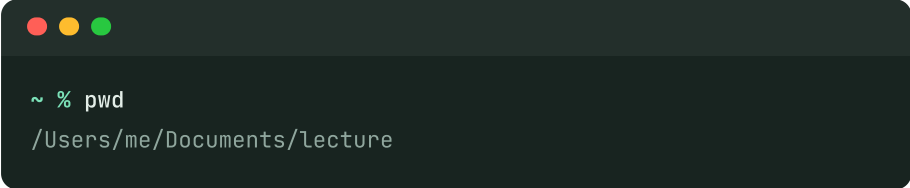
pwd

이동

print working directory

한마디로 **지금 내가 서 있는 폴더를 알려준다.**

형식 `pwd`



```
~ % pwd
/Users/me/Documents/lecture
```

유래

print working directory. "지금 작업 중인 폴더를 출력하라"는 뜻이다. 여기서 print가 프린터를 뜻하지 않는데도 굳이 print인 이유는, 텔레타이프 시절에는 결과가 정말로 종이에 인쇄되어 나왔기 때문이다.

주요 옵션

옵션	원래 단어	뜻
-P	<u>P</u> hysical	바로가기(링크)를 거쳐 들어왔다면, 바로가기 말고 실제 폴더의 위치를 보여준다.
-L	<u>L</u> ogical	바로가기를 거쳐 온 경로를 그대로 보여준다. 아무것도 안 붙이면 이것.

01 길을 잃었을 때

```
% pwd
```

프롬프트의 ~ 표시만으로 헛갈릴 때 전체 경로를 확인한다.

02 경로를 복사해 두기

```
% pwd | pbcopy
```

현재 경로가 클립보드에 들어간다. 메시저에 바로 붙여넣을 수 있다.

03 링크 뒤의 진짜 위치

```
% pwd -P
```

바로가기로 들어온 폴더라면 실제 폴더 경로가 나온다.

기억법

p-w-d를 "폴더 위치 답해"로 외워도 좋다. 원래 뜻은 print working directory.

꿀팁

Finder에서 폴더를 터미널 창에 끌어다 놓으면 그 폴더의 전체 경로가 자동으로 입력된다. pwd와 반대 방향의 요령이다.

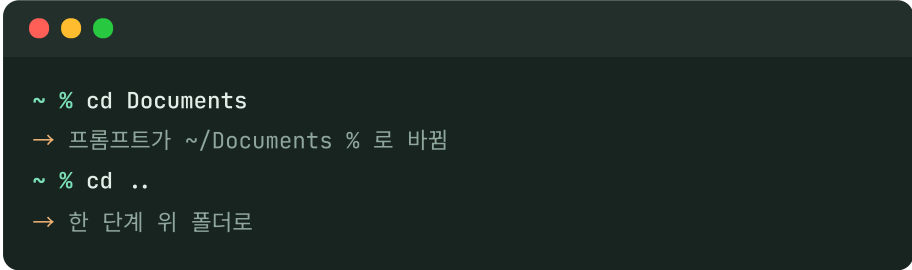
cd

이동

change directory

한마디로 **다른 폴더로 이동한다.**

형식 **cd [경로]**



```
~ % cd Documents
→ 프롬프트가 ~/Documents % 로 바뀜
~ % cd ..
→ 한 단계 위 폴더로
```

유래

change directory. 초기 유닉스 셸에서는 chdir이라는 조금 더 긴 이름이었다가, 가장 자주 치는 명령답게 두 글자 cd로 줄었다.

특별한 경로 기호

옵션	원래 단어	뜻
~	tilde = <u>home</u>	내 홈 폴더(/Users/내이름). cd ~는 "집으로 돌아가기".
..	parent	지금 폴더를 담고 있는 한 단계 위 폴더.
-	previous	바로 전에 있던 폴더. 두 폴더를 오갈 때 편하다.
/	root	디스크의 맨 꼭대기 폴더. 모든 폴더가 이 아래에 있다.

01 어디서든 집으로

```
% cd
```

아무것도 안 붙이면 홈 폴더로 돌아간다. cd ~와 같다.

02 두 폴더를 왔다 갔다

```
% cd -
```

직전 폴더로 돌아간다. 한 번 더 치면 다시 원래 자리로.

03 이름에 공백이 있을 때

```
% cd "My Projects"
```

따옴표로 감싸거나 My\ Projects처럼 공백 앞에 \를 붙인다.

04 두 단계 위로

```
% cd ../..
```

..를 /로 이어 붙이면 여러 단계를 한 번에 올라간다.

05 Tab으로 빠르게

```
% cd Doc + Tab
```

폴더 이름 앞 몇 글자만 치고 Tab을 누르면 나머지가 채워진다.

비하인드 스토리

cd는 프로그램 파일이 아니라 셸에 내장된 명령이다. 별도 프로그램으로 만들면 그 프로그램 안에서만 폴더가 바뀌고, 끝나는 순간 원래 자리로 돌아와 버리기 때문이다. "내 위치를 바꾸는 일은 나만 할 수 있다"는 유닉스 설계의 작은 철학이다.

꿀팁

Finder의 폴더를 터미널 창으로 끌어다 놓으면 경로가 입력된다. cd 한 칸 띄우고 폴더를 끌어다 놓으면 끝.

ls

이동

list

한마디로 **폴더 안에 뭐가 있는지 보여준다.**

형식 **ls** [옵션] [경로]

```
~ % ls -la
drwxr-xr-x  5 me  staff   160  9 29 09:03 .
-rw-r--r--  1 me  staff  1204  9 28 22:11 report.txt
-rw-----  1 me  staff    52  9 28 22:11 .env
```

주요 옵션

옵션	원래 단어	뜻
-l	<u>l</u> ong	한 줄에 하나씩, 권한·주인·크기·수정 날짜까지 자세히 보여준다.
-a	<u>a</u> ll	이름이 점(.)으로 시작하는 숨김 파일까지 모두 보여준다.
-h	<u>h</u> uman-readable	크기를 1048576 같은 바이트 숫자 대신 1.0M처럼 읽기 쉬운 단위로 보여준다. -l과 함께 쓴다.
-t	<u>t</u> ime	최근에 고친 파일이 맨 위에 오도록 정렬한다.
-S	<u>S</u> ize	용량이 큰 파일이 맨 위에 오도록 정렬한다.
-r	<u>r</u> everse	정렬 순서를 거꾸로 뒤집는다. 예: -ltr은 오래된 파일부터.
-R	<u>R</u> ecursive	하위 폴더 안의 내용까지 차례로 모두 보여준다.

맨 앞자리 읽는 법

d = 디렉토리, - = 일반 파일, l = 심볼릭 링크. 그 뒤 9자리는 소유자·그룹·기타 사용자 순서의 rwx 권한이다.

01 큰 파일부터 보기

```
% ls -lhS
```

크기(S) 순으로, 사람이 읽기 좋은 단위(h)로 보여준다.

02 최근에 고친 파일 5개

```
% ls -lt | head -5
```

| (파이프)는 앞 명령의 결과를 뒤 명령에 넘긴다. head는 앞부분만 자른다.

03 PNG 파일만

```
% ls *.png
```

이름이 .png로 끝나는 것만 보여준다.

04 폴더만 골라 보기

```
% ls -d */
```

끝이 /인 것, 즉 폴더만 나열한다.

05 한 줄에 하나씩

```
% ls -l
```

다른 명령으로 넘기거나 개수를 셀 때 편하다. 숫자 1이다.

비하인드 스토리

왜 숨김 파일은 마침표(.)로 시작할까? 초기 유닉스 개발자들이 ls 목록에서 현재 폴더(.)와 상위 폴더(..) 항목을 화면에 안 보이게 하려고, 이름이 마침표로 시작하는 항목은 건너뛰도록 임시로 짜둔 것이 시작이었다. 그런데 사람들이 이 허점을 이용해 설정 파일 이름 앞에 마침표를 붙여 "숨기기" 시작했고, 그것이 오늘날까지 이어진 숨김 파일 관례가 되었다 — 원래는 기능이 아니라 부작용이었다는 이야기다.

01 빠른 참조 · 이동과 둘러보기

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>pwd</code>	지금 위치 확인
<code>cd 폴더</code>	폴더로 이동
<code>cd ..</code>	한 단계 위로
<code>cd -</code>	직전 폴더로 되돌아가기
<code>cd</code>	홈 폴더로
<code>ls -lah</code>	숨김 파일까지 자세히
<code>ls -lt head</code>	최근 수정한 파일부터

파일과 디렉토리 다루기

폴더와 파일을 만들고, 지우고, 복사하고, 옮기고, 바로가기를 거는 일곱 가지 명령어.

mkdir

파일·디렉토리

make directory

한마디로 **새 폴더를 만든다.**

형식 **mkdir** [옵션] 디렉토리명

```
~ % mkdir photos
```

(출력 없음 - 성공하면 아무 말도 하지 않는 것이 유닉스 방식)

```
~ % mkdir -p 2026/09/lecture
```

→ 2026, 2026/09 폴더가 없어도 한 번에 전부 생성됨

유래

1970년대 초기 유닉스부터 있던 명령어. 그 시절 디렉토리는 특수한 형태의 파일이라 `mknod` 같은 저수준 명령으로도 만들 수 있었지만, 실수 위험이 커서 안전하게 디렉토리만 만드는 전용 명령으로 따로 뗐다.

주요 옵션

옵션	원래 단어	뜻
-p	parents	중간 폴더가 없어도 알아서 함께 만든다. a/b/c를 만들 때 a, b가 없어도 된다.
-m	mode	폴더를 만들면서 권한(누가 읽고 쓸 수 있는지)도 함께 정한다. 예: -m 700
-v	verbose	폴더를 하나 만들 때마다 만든 이름을 화면에 알려준다.

상황별 예제

01 폴더 여러 개를 한 번에

```
% mkdir css js images
```

세 폴더가 동시에 생긴다. 이름을 공백으로 나열하면 된다.

02 프로젝트 뼈대를 한 줄로

```
% mkdir -p project/{src,docs,tests}
```

중괄호 안의 이름마다 폴더가 생긴다. project/src, project/docs, project/tests.

03 오늘 날짜로 백업 폴더

```
% mkdir -p backup/$(date +%Y-%m-%d)
```

backup/2026-09-30 생성. \$()는 안쪽 명령의 결과를 그 자리에 끼워 넣는다.

04 이름에 띄어쓰기가 있을 때

```
% mkdir "강의 자료"
```

따옴표가 없으면 '강의'와 '자료' 두 폴더가 생겨버린다.

05 만들고 바로 들어가기

```
% mkdir notes && cd notes
```

&&는 앞 명령이 성공했을 때만 뒤 명령을 실행한다.

기억법

make + directory를 줄이면 그대로 mkdir. "메이크 디렉토리"를 빠르게 이어 읽으면 명령어 이름이 된다.

꿀팁

-p 없이 mkdir a/b를 실행했는데 a가 없으면 에러가 난다. 중첩 폴더를 만들 땐 습관적으로 -p를 붙이면 안전하다.

rmdir

파일·디렉토리

remove directory

한마디로 **빈 폴더만 지운다.**

형식 **rmdir** 디렉토리명

```
~ % rmdir empty_folder
(비어 있으면 조용히 삭제)
~ % rmdir photos
rmdir: photos: Directory not empty
```

유래

remove + directory. mkdir의 반대 짝. 오직 "완전히 빈 폴더"만 지울 수 있도록 설계해, 실수로 내용물 가득한 폴더를 날리는 사고를 원천적으로 막는다.

rm -r 과의 차이

rmdir	빈 폴더만 삭제 가능, 안전
rm -r	내용물 포함 통째로 삭제, 위험

상황별 예제

01 빈 폴더 여러 개 정리

```
% rmdir old1 old2 old3
```

셋 다 비어 있으면 한 번에 사라진다.

02 겹겹이 빈 폴더 거슬러 지우기

```
% rmdir -p a/b/c
```

c를 지우고, 비었으면 b, 다시 비었으면 a까지 올라가며 지운다.

03 비어 보이는데 안 지워질 때

```
% ls -A photos
```

Finder가 몰래 만든 .DS_Store 같은 숨김 파일이 보이면 그 때문이다.

기억법

rm(지우기) + dir(디렉토리). "빈 방만 치울 수 있다"로 기억하면 rm -r과 헷갈리지 않는다.

비하인드 스토리

rm -r 하나로 통합할 수도 있었지만 유닉스 설계자들은 굳이 rmdir을 따로 남겨뒀다. "위험한 동작일수록 명시적으로 요청해야 한다"는 초기 유닉스의 설계 철학이 명령어 하나에도 그대로 스며 있다.

touch

파일·디렉토리

touch (살짝 건드리다)

한마디로 **빈 파일을 만들거나 수정 시각을 지금으로 바꾼다.**

형식 **touch** 파일명

```
~ % touch index.html
~ % ls -l index.html
-rw-r--r--  1 me  staff   0  9 30 10:02 index.html
```

유래

touch는 "살짝 건드리다"라는 뜻이다. 원래 목적은 파일을 열지 않고 살짝 건드려 수정 시각만 지금으로 바꾸는 것이었다. 파일이 없으면 새로 만들어 주는 부수 효과가 오히려 더 유명해졌다.

주요 옵션

옵션	원래 단어	뜻
-c	no- <u>c</u> reate	파일이 없으면 새로 만들지 않는다. 있는 파일의 시각만 바꾸고 싶을 때.
-t	<u>t</u> ime	수정 시각을 원하는 날짜로 바꾼다. 예: 202601010900 = 2026년 1월 1일 9시
-a	<u>a</u> ccess time	마지막으로 파일을 연 시각만 바꾼다.
-m	<u>m</u> odification time	마지막으로 파일을 고친 시각만 바꾼다.

01 파일 여러 개 한 번에

```
% touch index.html style.css app.js
```

크기 0짜리 빈 파일 세 개가 생긴다.

02 번호 붙은 파일 만들기

```
% touch page{1..5}.md
```

page1.md부터 page5.md까지 한꺼번에 생긴다.

03 빈 폴더를 git에 올리려면

```
% touch assets/.gitkeep
```

git은 빈 폴더를 기록하지 않아서, 이름뿐인 파일을 하나 넣어 두는 관례다.

04 날짜를 과거로 되돌리기

```
% touch -t 202601010900 memo.txt
```

수정 시각이 2026년 1월 1일 9시로 바뀐다.

비하인드 스토리

빌드 도구 make는 파일의 수정 시각을 보고 다시 컴파일할지 결정한다. 그래서 개발자들은 내용은 그대로 두고 touch로 파일을 "건드려" 강제로 다시 빌드시키곤 했다. 이름이 기능을 그대로 설명하는 드문 명령어다.

rm

파일·디렉토리

remove

한마디로 **파일과 폴더를 휴지통 없이 지운다.**

형식 **rm [옵션] 파일명**

```
~ % rm draft.txt
(휴지통을 거치지 않고 즉시 삭제)
~ % rm -r old_project/
→ 폴더와 내용물 전부 삭제
```

주요 옵션

옵션	원래 단어	뜻
-r	<u>r</u> ecursive	폴더와 그 안의 파일·하위 폴더까지 전부 지운다. 폴더를 지울 때 필요하다.
-f	<u>f</u> orce	"정말 지을까요?"를 묻지 않는다. 없는 파일이어도 오류 없이 넘어간다.
-i	<u>i</u> nteractive	파일마다 "지을까요?"라고 묻고, y를 눌러야 지운다.
-v	<u>v</u> erbose	지운 파일 이름을 하나씩 화면에 보여준다.

유래

remove를 그대로 줄인 이름. Finder의 휴지통과 달리 완전히 즉시 삭제되며, 복구 경로가 사실상 없다는 점이 가장 중요한 특징이다.

상황별 예제

01 여러 파일 한꺼번에

```
% rm draft1.txt draft2.txt
```

나열한 파일이 모두 삭제된다.

02 특정 확장자 전부

```
% rm *.log
```

*는 '아무 글자나'라는 뜻. .log로 끝나는 파일이 모두 지워진다.

03 하나씩 확인하며 지우기

```
% rm -i *.jpg
```

파일마다 remove? 라고 묻고, y를 눌러야 지운다.

04 이름이 -로 시작하는 파일

```
% rm -- -draft.txt
```

-- 뒤는 옵션이 아니라 파일 이름이라는 약속이다.

05 휴지통으로 보내고 싶다면

```
% mv report.txt ~/.Trash/
```

rm 대신 Finder 휴지통으로 옮기면 나중에 되살릴 수 있다.

꿀팁

삭제 대상이 넓은 명령(`rm -rf 폴더/*`)을 실행하기 전엔 같은 경로로 `ls`를 먼저 실행해 무엇이 지워질지 눈으로 확인하는 습관이 사고를 막아준다.

비하인드 스토리

2015년 스팀(Steam) 리눅스 클라이언트에는 `rm -rf "$STEAMROOT/*"` 라는 코드가 있었는데, 특정 조건에서 STEAMROOT 변수가 빈 값이 되면서 사실상 `rm -rf /*` 처럼 동작해 사용자 홈 폴더 전체를 삭제해버린 사건이 있었다. 변수 하나, 따옴표 하나가 시스템 전체를 위협할 수 있다는 것을 보여준 유명한 사고로 지금도 개발자들 사이에서 경고 사례로 회자된다.

cp

파일·디렉토리

copy

한마디로 복사본을 만든다. 원본은 그대로.

형식 cp 원본 대상

```
~ % cp report.txt backup.txt
→ report.txt는 그대로 있고 backup.txt가 새로 생김
~ % cp -r project/ project_backup/
→ 폴더 전체가 복사됨
```

주요 옵션

옵션	원래 단어	뜻
-R	<u>R</u> ecursive	폴더를 안의 내용까지 통째로 복사한다. 폴더를 복사할 때 필수다. 소문자 -r도 같다.
-i	<u>i</u> nteractive	같은 이름의 파일이 이미 있으면 덮어쓸지 먼저 묻는다.
-n	<u>n</u> o-clobber	같은 이름의 파일이 이미 있으면 복사하지 않고 그대로 둔다.
-v	<u>v</u> erbose	복사한 파일 이름을 하나씩 보여준다.
-p	<u>p</u> reserve	권한과 수정 날짜까지 원본과 똑같이 복사한다. 없으면 날짜가 복사한 시각으로 바뀐다.

유래

copy를 그대로 줄인 이름. 이름에 특별한 사연은 없지만, mv와 짝을 이루는 가장 기본적인 파일 조작 명령이다.

01 다른 폴더에 같은 이름으로

```
% cp report.txt ~/Documents/
```

대상이 폴더면 원래 이름 그대로 그 안에 복사된다.

02 사진만 골라 복사

```
% cp *.jpg photos/
```

.jpg 파일 전부가 photos 폴더로 복사된다.

03 수정 전 백업본 만들기

```
% cp config.json config.json.bak
```

고치다 망가지면 .bak 파일로 되돌리면 된다.

04 있으면 건드리지 않기

```
% cp -n template.md README.md
```

README.md가 이미 있으면 아무 일도 일어나지 않는다.

05 폴더 속 내용물만 복사

```
% cp -R src/ dest/
```

macOS에서는 끝의 / 유무가 중요하다. src/는 내용물만, src는 폴더째 복사된다.

기억법

cp는 원본과 사본, 둘 다 남는다. mv는 하나만 남는다 — 이 대비로 두 명령을 함께 외우면 절대 안 헛갈린다.

꿀팁

대상 파일이 이미 있으면 확인 없이 바로 덮어쓴다. 중요한 파일을 다룰 땐 cp -i 습관을 들이자.

mv

파일·디렉토리

move

한마디로 옮기거나 이름을 바꾼다. 원본은 사라진다.

형식 mv 원본 대상

```
~ % mv old.txt new.txt
→ 이름만 바뀜(같은 위치)
~ % mv report.txt ~/Desktop/
→ Desktop 폴더로 이동, 원본 위치엔 남지 않음
```

유래

move를 그대로 줄인 이름. cp와 달리 폴더 이동에도 -r 옵션이 필요 없다.

주요 옵션

옵션	원래 단어	뜻
-i	<u>i</u> nteractive	같은 이름의 파일이 이미 있으면 덮어쓸지 먼저 묻는다.
-n	<u>n</u> o-clobber	같은 이름의 파일이 이미 있으면 옮기지 않는다.
-f	<u>f</u> orce	묻지 않고 바로 덮어쓴다.
-v	<u>v</u> erbose	옮긴 파일 이름을 하나씩 보여준다.

상황별 예제

01 확장자 바꾸기

```
% mv notes.txt notes.md
```

내용은 그대로, 이름표만 바뀐다.

02 PDF만 모아서 이동

```
% mv *.pdf ~/Documents/
```

현재 폴더의 PDF가 전부 문서 폴더로 옮겨진다.

03 한 단계 위 폴더로

```
% mv photo.jpg ..
```

..는 '상위 폴더'를 뜻하는 약속이다.

04 덮어쓰기 사고 방지

```
% mv -n new.txt backup/
```

backup/에 같은 이름이 있으면 옮기지 않는다.

비하인드 스토리

같은 디스크(파일시스템) 안에서 mv는 실제로 파일 내용을 복사하지 않는다. 폴더 목차에 적힌 위치 정보만 바꿔치기하기 때문에 수십 기가바이트짜리 파일도 순식간에 "이동"된다. 하지만 외장하드처럼 다른 파티션으로 옮길 땐 내부적으로 복사 후 원본 삭제가 일어나 시간이 걸린다 — 같은 mv인데 속도 차이가 나는 이유다.

ln

파일·디렉토리

link

한마디로 **파일이나 폴더의 바로가기**를 만든다.

형식 **ln -s 원본 링크이름**

```
~ % ln -s ~/Dropbox/notes ~/notes
~ % ls -l ~/notes
lrwxr-xr-x  1 me  staff   27 ... notes ->
/Users/me/Dropbox/notes
```

유래

link를 줄인 이름. -s를 붙이면 심볼릭 링크(바로가기), 붙이지 않으면 하드 링크(같은 파일에 이름을 하나 더 붙이기)가 된다.

주요 옵션

옵션	원래 단어	뜻
-s	<u>s</u> ymbolic	원본을 가리키는 바로가기 파일을 만든다. 거의 항상 이 옵션을 쓴다.
-f	<u>f</u> orce	같은 이름의 파일이나 링크가 이미 있으면 지우고 새로 만든다.
-v	<u>v</u> erbose	만든 링크를 화면에 보여준다.

상황별 예제

01 자주 가는 깊은 폴더를 가까이

```
% ln -s ~/Library/Mobile\ Documents/com~apple~CloudDocs ~/iCloud
```

iCloud Drive처럼 깊은 폴더를 홈에서 바로 연다.

02 설정 파일 한곳에서 관리

```
% ln -s ~/dotfiles/.zshrc ~/.zshrc
```

설정을 git 폴더에 두고 링크만 걸어 여러 컴퓨터에서 공유한다.

03 링크가 가리키는 곳 확인

```
% ls -l ~/notes
```

-> 뒤에 원본 경로가 표시된다.

04 링크만 지우기

```
% rm ~/notes
```

원본은 안전하다. 단, 끝에 /를 붙여 `rm -r ~/notes/` 로 지우면 원본 내용이 지워질 수 있다.

기억법

순서는 cp와 같다. "진짜 먼저, 가짜 나중". `ln -s` 원본 바로가기.

비하인드 스토리

Finder에서 만드는 "가상본(alias)"은 이름은 비슷하지만 터미널의 심볼릭 링크와 다른 물건이다. Finder 가상본으로는 `cd`로 들어갈 수 없다. 터미널에서 쓸 바로가기는 `ln -s`로 만들어야 한다.

02 빠른 참조 · 파일과 디렉토리 다루기

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>mkdir -p a/b/c</code>	중첩 폴더를 한 번에
<code>touch</code> 파일	빈 파일 만들기
<code>rmdir</code> 폴더	빈 폴더 지우기
<code>rm -i</code> 파일	확인하며 삭제
<code>rm -r</code> 폴더	폴더째 삭제 (되돌릴 수 없음)
<code>cp -R</code> 원본/ 대상/	폴더 내용물 복사
<code>mv</code> 옛이름 새이름	이름 바꾸기
<code>mv *.pdf</code> 폴더/	여러 파일 한꺼번에 이동
<code>ln -s</code> 원본 바로가기	바로가기 만들기

파일을 열지 않고 터미널 안에서 내용을 읽는 방법. 짧으면 cat, 길면 less, 앞뒤만 보려면 head와 tail.

cat

내용 보기

concatenate

한마디로 **파일 내용을 화면에 쏟아낸다.**

형식 **cat** 파일명

```
~ % cat hello.txt
```

안녕하세요.

터미널에 오신 것을 환영합니다.

유래

concatenate(사슬처럼 이어 붙이다)에서 왔다. 원래 목적은 여러 파일을 이어 붙이는 것이었고, 파일 하나만 주면 그냥 내용을 보여주게 된다. 고양이(cat)와는 아무 관계가 없지만, 덕분에 개발자 굿즈에 고양이가 자주 등장한다.

주요 옵션

옵션	원래 단어	뜻
-n	<u>n</u> umber	모든 줄 앞에 1, 2, 3... 줄 번호를 붙여 보여준다.
-b	num <u>b</u> er non-blank	줄 번호를 붙이되 빈 줄은 건너뛴다.
-s	<u>s</u> queeze	빈 줄이 여러 개 이어지면 한 줄로 줄여 보여준다.

01 두 파일을 하나로 합치기

```
% cat part1.txt part2.txt > full.txt
```

이름 그대로 이어 붙이는 원래 용도다. >는 결과를 파일로 저장.

02 줄 번호와 함께 보기

```
% cat -n script.sh
```

오류 메시지가 "12번째 줄"을 가리킬 때 찾기 쉽다.

03 짧은 메모 바로 쓰기

```
% cat > memo.txt
```

내용을 입력하고 Control+D를 누르면 저장된다.

04 숨은 설정 파일 확인

```
% cat ~/.zshrc
```

지금 적용된 zsh 설정을 한눈에 본다.

꿀팁

수백 줄이 넘는 파일을 cat으로 열면 화면이 순식간에 지나가 버린다. 긴 파일은 less로 여는 편이 낫다.

비하인드 스토리

cat file | grep 단어 처럼 굳이 cat을 거치는 습관은 개발자들 사이에서 "쓸모없는 cat 사용상(Useless Use of Cat Award)"이라는 농담의 대상이 됐다. grep 단어 file 한 번이면 충분하기 때문이다.

less

내용 보기

less is more

한마디로 긴 파일을 한 화면씩 넘겨 본다.

형식 **less** 파일명

```
~ % less app.log
```

→ 한 화면씩 표시, Space로 넘기고 q로 종료

유래

예전부터 한 화면씩 끊어 보여주는 more라는 명령이 있었다. 1980년대 중반 마크 누델먼이 뒤로 되돌아가기까지 되는 개량판을 만들며 "less is more(적은 것이 더 많다)"라는 말장난으로 이름을 less라 붙였다.

less 안에서 쓰는 키

옵션	원래 단어	뜻
Space / b	forward · <u>b</u> ack	Space는 다음 화면으로, b는 이전 화면으로 넘긴다.
/단어	search	입력한 단어가 있는 곳으로 이동한다.
n / N	<u>n</u> ext	같은 단어의 다음 위치(n), 이전 위치(N)로 이동한다.
g / G	<u>g</u> o	파일의 맨 처음(g), 맨 끝(G)으로 이동한다.
F	<u>F</u> ollow	파일에 새 내용이 추가되면 계속 따라가며 보여준다. Control+C로 멈춘다.
q	<u>q</u> uit	less를 닫고 터미널로 돌아온다.

01 로그를 실시간으로

```
% less +F app.log
```

tail -f처럼 새 줄을 따라간다. Control+C로 멈추면 자유롭게 스크롤할 수 있다.

02 긴 명령 결과 넘겨 보기

```
% ls -la /usr/bin | less
```

결과가 화면을 넘어갈 때 뒤에 | less를 붙이는 습관.

03 줄 번호와 함께

```
% less -N config.yml
```

왼쪽에 줄 번호가 붙는다.

04 검색하며 읽기

```
% /ERROR
```

less 화면에서 /를 누르고 단어를 치면 해당 위치로 이동한다.

기억법

more보다 기능이 많은데 이름은 less. "less is more" 농담만 기억하면 이름이 저절로 떠오른다.

꿀팁

man 명령으로 여는 설명서 화면도 사실 less다. 그래서 설명서 안에서도 /검색과 q 종료가 똑같이 통한다.

head

내용 보기

head (머리)

한마디로 **파일의 앞부분만 보여준다.**

형식 **head [-n 줄수] 파일명**

```
~ % head -n 3 data.csv
name,age,city
kim,31,Seoul
Lee,28,Busan
```

유래

head, 즉 머리. 파일의 머리 부분(앞 10줄)을 보여준다. 꼬리를 보여주는 tail과 짝을 이룬다.

주요 옵션

옵션	원래 단어	뜻
-n	n umber of lines	앞에서부터 몇 줄을 볼지 정한다. 적지 않으면 10줄.
-c	bytes (c haracters)	줄 대신, 앞에서부터 몇 바이트(글자 크기 단위)를 볼지 정한다.

상황별 예제

01 CSV의 제목 줄만

```
% head -1 data.csv
```

어떤 열이 있는지 파일을 열지 않고 확인한다.

02 최근 파일 5개

```
% ls -lt | head -5
```

다른 명령 결과의 앞부분만 자를 때 가장 많이 쓴다.

03 거대한 파일 맛보기

```
% head -n 20 huge.log
```

수 GB짜리 파일도 앞 20줄만 순식간에 본다.

기억법

head는 머리, tail은 꼬리. 파일을 동물이라고 생각하면 절대 안 헛갈린다.

tail

내용 보기

tail (꼬리)

한마디로 **파일의 끝부분만 보여준다. 실시간 추적도 된다.**

형식 **tail [-n 줄수] 파일명**

```
~ % tail -f server.log
10:02:11 GET /index.html 200
10:02:14 GET /api/users 200
→ 새 줄이 생길 때마다 계속 출력, Control+C로 종료
```

유래

tail, 즉 꼬리. 파일의 마지막 10줄을 보여준다. -f(follow) 옵션 덕분에 서버 관리자들이 가장 오래 쳐두는 명령이 되었다.

주요 옵션

옵션	원래 단어	뜻
-n	n umber of lines	끝에서부터 몇 줄을 볼지 정한다. 적지 않으면 10줄. +2처럼 쓰면 '2번째 줄부터 끝까지'.
-f	f ollow	끝내지 않고 기다리다가, 파일에 새 줄이 생기면 바로 보여준다.
-F	F ollow (retry)	-f와 같지만, 로그 파일이 새 파일로 바뀌어도 계속 따라간다.
-r	r everse	마지막 줄부터 거꾸로 보여준다.

01 마지막 20줄

```
% tail -n 20 app.log
```

방금 무슨 일이 있었는지 확인할 때.

02 에러만 실시간으로

```
% tail -f app.log | grep ERROR
```

로그가 쌓이는 동안 ERROR가 들어간 줄만 걸러 본다.

03 제목 줄 빼고 전부

```
% tail -n +2 data.csv
```

+2는 "2번째 줄부터"라는 뜻. CSV 데이터만 뽑을 때 유용하다.

꿀팁

tail -f 로그 | grep 키워드 는 서버 문제를 추적할 때 가장 많이 쓰는 조합이다.
외워 둘 가치가 있다.

echo

내용 보기

echo (메아리)

한마디로 **입력한 말을 그대로 되돌려준다.**

형식 **echo** 문자열

```
~ % echo "안녕, 터미널"
안녕, 터미널
~ % echo $HOME
/Users/me
```

유래

echo, 메아리. 외친 말을 그대로 돌려주는 메아리처럼, 받은 글자를 화면에 그대로 출력한다. 단순해 보이지만 변수 확인과 파일 쓰기에 매일 쓰인다.

주요 옵션

옵션	원래 단어	뜻
-n	<u>n</u> o newline	출력 끝에 줄바꿈을 붙이지 않는다. 다음 출력이 같은 줄에 이어진다.
>	redirect	출력을 파일에 저장한다. 파일에 있던 원래 내용은 지워진다.
>>	append	출력을 파일 맨 끝에 덧붙인다. 원래 내용은 그대로 남는다.

상황별 예제

01 변수 값 확인

```
% echo $PATH
```

프로그램을 찾는 경로 목록이 나온다.

02 파일에 한 줄 쓰기

```
% echo "첫 줄" > note.txt
```

note.txt가 없으면 만들고, 있으면 내용을 덮어쓴다.

03 파일 끝에 덧붙이기

```
% echo "둘째 줄" >> note.txt
```

기존 내용은 그대로 두고 아래에 추가한다.

04 방금 명령이 성공했나

```
% echo $?
```

0이면 성공, 그 외 숫자면 실패다.

꿀팁

> 는 덮어쓰기, >> 는 덧붙이기. 한 글자 차이로 파일 내용이 통째로 사라질 수 있으니 설정 파일에 쓸 때는 특히 >>인지 확인하자.

03 빠른 참조 · 내용 들여다보기

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>cat</code> 파일	짧은 파일 내용 보기
<code>cat a b > c</code>	두 파일 이어 붙이기
<code>less</code> 파일	긴 파일 넘겨 보기 (q 종료)
<code>head -n 5</code> 파일	앞 5줄
<code>tail -n 20</code> 파일	끝 20줄
<code>tail -f</code> 로그	로그 실시간 추적
<code>echo "글" >> 파일</code>	파일 끝에 한 줄 덧붙이기

파일 자체를 찾을 땐 `find`, 파일 속 글자를 찾을 땐 `grep`, 명령어의 정체를 찾을 땐 `which`.

find

검색

find

한마디로 조건에 맞는 파일을 찾아낸다.

형식 `find` 검색경로 -조건 값

```
~ % find . -name "*.txt"
./notes/report.txt
./draft.txt
~ % find . -mtime -7
→ 최근 7일 이내 수정된 파일만 표시
```

주요 옵션

옵션	원래 단어	뜻
-name	<u>name</u>	파일 이름으로 찾는다. *는 '아무 글자나'라는 뜻이다. 예: "*.txt". 대소문자를 구분한다.
-iname	<u>i</u> nsensitive name	-name과 같지만 대소문자를 가리지 않는다. Photo.JPG와 photo.jpg를 모두 찾는다.
-type	<u>type</u> (f= <u>f</u> ile, d= <u>d</u> irectory)	f를 붙이면 파일만, d를 붙이면 폴더만 찾는다.
-size	<u>size</u>	크기로 찾는다. +500M은 500MB보다 큰 것, -1k는 1KB보다 작은 것.
-mtime	<u>m</u> odification <u>t</u> ime	고친 날짜로 찾는다. -7은 7일 이내, +30은 30일보다 오래된 것.
-exec	<u>execute</u>	찾은 파일마다 뒤에 적은 명령을 실행한다. {}는 찾은 파일, \;는 명령의 끝.

삭제와 조합

`find . -name "*.log" -delete` 처럼 삭제와 묶어 쓸 수 있다. 되돌릴 수 없으니 -delete를 빼고 먼저 실행해 어떤 파일이 걸리는지 확인한 뒤 붙이는 게 안전하다.

상황별 예제

01 용량 먹는 범인 찾기

```
% find ~ -size +500M
```

홈 폴더 전체에서 500MB가 넘는 파일을 찾는다.

02 특정 이름의 폴더만

```
% find . -type d -name "node_modules"
```

흩어져 있는 node_modules 폴더 위치를 한 번에 파악한다.

03 macOS 찌꺼기 청소

```
% find . -name ".DS_Store" -delete
```

Finder가 만든 .DS_Store 파일을 모두 지운다.

04 찾은 파일을 바탕화면으로

```
% find . -name "*.png" -exec cp {} ~/Desktop/ \;
```

{ }는 찾은 파일 하나하나, \;는 명령의 끝을 뜻한다.

05 현재 폴더만 알게

```
% find . -maxdepth 1 -name "*.md"
```

하위 폴더까지 파고들지 않고 지금 폴더만 본다.

기억법

이름 그대로 find = 찾다. grep과 헷갈릴 땐 "find는 파일을, grep은 내용을 찾는다"로 구분한다.

grep

검색

global regular expression print

한마디로 **파일 속 글자를 찾아낸다.**

형식 **grep "검색어" 파일명**

```
~ % grep -rni "todo" src/  
src/App.tsx:42:// TODO: 강의 3장 슬라이드 추가  
src/utils.ts:8:// todo: 정렬 함수 리팩토링
```

주요 옵션

옵션	원래 단어	뜻
-r	<u>r</u> ecursive	지정한 폴더와 그 아래 모든 파일에서 찾는다.
-i	<u>i</u> gnore case	대문자와 소문자를 같은 글자로 본다. Error, error, ERROR를 모두 찾는다.
-n	line <u>n</u> umber	찾은 줄이 파일의 몇 번째 줄인지 번호를 붙여 보여준다.
-v	in <u>v</u> ert match	거꾸로, 그 글자가 없는 줄만 보여준다.
-l	<u>l</u> ist files	내용은 빼고, 그 글자가 들어 있는 파일의 이름만 보여준다.
-c	<u>c</u> ount	찾은 줄이 몇 개인지 숫자만 알려준다.
-w	<u>w</u> ord	하나의 단어로 딱 떨어질 때만 찾는다. cat을 찾을 때 category는 빠진다.
-C	<u>C</u> ontext	찾은 줄의 앞뒤 몇 줄을 함께 보여준다. 예: -C 2는 앞뒤 2줄씩.
-E	<u>E</u> xtended regex	a b(a 또는 b)처럼 더 풍부한 검색식을 쓸 수 있게 한다.

유래

"global regular expression print"의 줄임말. 옛 텍스트 편집기 ed에서 쓰던 g/re/p (전역/정규식/출력) 명령 조합에서 이름이 나왔다.

상황별 예제

01 에러가 몇 번 났나

```
% grep -c "error" app.log
```

error가 들어간 줄의 개수만 숫자로 알려준다.

02 커밋 전 비밀키 점검

```
% grep -rL "API_KEY" .
```

API_KEY가 적힌 파일 이름만 뽑는다. 올리면 안 될 파일을 미리 찾는다.

03 앞뒤 사정까지 보기

```
% grep -n -C 2 "Exception" app.log
```

일치한 줄의 앞뒤 2줄을 줄 번호와 함께 보여준다.

04 둘 중 하나라도

```
% grep -E "error|warn" app.log
```

|는 '또는'. error나 warn이 있는 줄을 모두 찾는다.

05 예전에 친 명령 되찾기

```
% history | grep ssh
```

입력 기록에서 ssh가 들어간 명령만 걸러낸다.

06 단어로만 찾기

```
% grep -w "cat" notes.txt
```

category나 concat은 빼고 cat이라는 단어만 찾는다.

비하인드 스토리

벨 연구소의 더그 매킬로이가 켄 톰슨에게 "ed 편집기의 전역 검색 기능만 따로 떼어 독립 프로그램으로 만들어 달라"고 부탁했고, 톰슨이 하룻밤 만에 만들어 다음 날 아침 내놓았다는 일화가 전해진다. "작은 일을 잘하는 작은 도구를 여러 개 만들어 조합한다"는 유닉스 철학을 가장 잘 보여주는 사례로 꼽힌다.

which

검색

which (어느 것)

한마디로 그 명령어가 실제로 어디 있는 프로그램인지 알려준다.

형식 **which** 명령어

```
~ % which python3
/usr/bin/python3
```

유래

which, "어느 것". 같은 이름의 프로그램이 여러 곳에 설치돼 있을 때 지금 실행되는 게 어느 것인지 알려준다.

주요 옵션

옵션	원래	단어	뜻
----	----	----	---

-a	a ll		같은 이름의 프로그램이 여러 곳에 있으면 전부 보여준다. 맨 위 것이 실제로 실행된다.
-----------	-------------	--	--

상황별 예제

01 설치가 됐나 확인

```
% which node
```

아무것도 나오지 않으면 설치가 안 됐거나 PATH에 없는 것이다.

02 여러 버전이 섞였을 때

```
% which -a python3
```

/opt/homebrew/bin/python3와 /usr/bin/python3가 둘 다 보이면, 위에 있는 것이 실행된다.

03 별명인지 확인

```
% which ll
```

zsh에서는 ll: aliased to ls -lah 처럼 별명도 알려준다.

꿀팁

"분명 설치했는데 command not found"가 뜨면 which로 먼저 확인하자. 설치 위치가 PATH에 빠져 있는 경우가 대부분이다. 해결은 export 항목에 있다.

04 빠른 참조 · 찾고 뒤지기

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

```
find . -name "*.txt"
```

이름으로 파일 찾기

```
find ~ -size +500M
```

큰 파일 찾기

```
grep -rn "단어" .
```

폴더 전체에서 글자 찾기

```
grep -rl "단어" .
```

그 글자가 든 파일 이름만

```
which 명령
```

명령의 실제 위치

```
which -a python3
```

같은 이름이 여러 개인지

세고, 정렬하고, 바꾸고, 넘겨주는 명령어. 파이프(|)로 이어 붙이면 한 줄짜리 작은 프로그램이 된다.

WC

텍스트 가공

word count

한마디로 줄, 단어, 글자 수를 센다.

형식 WC [옵션] 파일명

```
~ % wc essay.txt
      42      318     2204 essay.txt
→ 줄 42 · 단어 318 · 바이트 2204
```

유래

word count. 이름은 단어 세기지만 줄과 바이트까지 한 번에 세어 준다.

주요 옵션

옵션	원래 단어	뜻
-l	<u>l</u> ines	몇 줄인지만 센다.
-w	<u>w</u> ords	띄어쓰기로 나뉜 단어가 몇 개인지만 센다.
-c	bytes (<u>c</u> haracters)	파일 크기를 바이트로 센다. 한글은 한 글자가 3바이트라 글자 수와 다르다.
-m	<u>m</u> ultibyte characters	한글을 포함한 실제 글자 수를 센다.

01 데이터가 몇 줄인가

```
% wc -l data.csv
```

CSV라면 제목 줄 포함 행 개수다.

02 파일이 몇 개인가

```
% ls | wc -l
```

ls 결과를 한 줄에 하나씩 세면 파일 개수가 된다.

03 코드가 몇 줄인가

```
% cat src/*.ts | wc -l
```

프로젝트 규모를 가늠할 때 쓴다.

04 한글 글자 수 세기

```
% wc -m essay.txt
```

한글은 한 글자가 3바이트라 -c로 세면 세 배로 나온다. 글자 수는 -m.

꿀팁

원고 분량을 셀 때 wc -m이 유용하다. 공백과 줄바꿈도 한 글자로 세므로 "공백 포함 글자 수"와 같다.

sort

텍스트 가공

sort (정렬하다)

한마디로 줄을 순서대로 정렬한다.

형식 **sort** [옵션] 파일명

```
~ % sort fruits.txt
apple
banana
cherry
```

유래

sort, 정렬하다. 기본은 글자 순서(사전 순)라서 숫자를 정렬할 때는 옵션이 필요하다.

주요 옵션

옵션	원래 단어	뜻
-n	<u>n</u> umeric	글자가 아닌 숫자 크기로 정렬한다. 없으면 10이 2보다 앞에 온다.
-r	<u>r</u> everse	순서를 거꾸로 뒤집는다. 큰 것, 나중 것이 위로 온다.
-h	<u>h</u> uman numeric	1K, 2M, 3G처럼 단위가 붙은 크기를 제대로 비교해 정렬한다.
-u	<u>u</u> nique	똑같은 줄이 여러 개면 하나만 남긴다.
-k	<u>k</u> ey	몇 번째 칸(열)을 기준으로 정렬할지 정한다. 예: -k2
-t	field separa <u>t</u> or	칸을 나누는 기호를 정한다. 예: CSV는 -t,

01 숫자를 제대로 정렬

```
% sort -n scores.txt
```

-n이 없으면 10이 2보다 앞에 온다. 글자 "1"이 "2"보다 앞이기 때문.

02 중복 없이 정리

```
% sort -u emails.txt
```

같은 이메일이 여러 번 있어도 한 번씩만 남는다.

03 가장 많이 나온 순위

```
% sort access.log | uniq -c | sort -rn | head
```

같은 줄끼리 모으고(sort), 개수를 세고(uniq -c), 많은 순으로 다시 정렬한다.

04 CSV 둘째 열 기준

```
% sort -t, -k2 data.csv
```

쉼표(,)로 열을 나누고 2번째 열로 정렬한다.

05 용량 순 정렬

```
% du -sh * | sort -rh
```

-h는 1.2G와 48M 같은 단위를 이해한다.

비하인드 스토리

짜깁 명령 uniq는 "바로 붙어 있는" 중복만 지운다. 그래서 거의 언제나 sort | uniq 순서로 함께 쓰인다. 두 글자로 말하는 사람들의 문장이 파이프(|)로 이어지는 대표적인 예다.

sed

텍스트 가공

stream editor

한마디로 **파일 속 글자를 한꺼번에 바꾼다.**

형식 **sed 's/찾을말/바꿀말/g' 파일명**

```
~ % sed 's/2025/2026/g' notice.txt
2026년 강의 일정 안내
→ 결과만 화면에 출력, 원본 파일은 그대로
```

유래

stream editor. 흘러가는 글자(stream)를 편집한다는 뜻이다. 1970년대 벨 연구소의 리 맥마흔이 ed 편집기의 명령을 파이프 속에서 쓸 수 있게 만든 것이 시작이다.

주요 옵션

옵션	원래 단어	뜻
s	s ubstitute	찾을 글자를 바꿀 글자로 바꾸는 명령. s/찾을말/바꿀말/ 형태로 쓴다.
g	g lobal	한 줄에 같은 글자가 여러 번 있어도 모두 바꾼다. 없으면 줄마다 첫 번째만 바꾼다.
-i	i n-place	결과를 화면에 보여주는 대신 원본 파일을 직접 고친다. macOS에서는 -i "로 쓴다.
-n + p	n o print + p rint	평소 출력을 끄고(-n), p로 지정한 줄만 보여준다. 예: -n '10,20p'
d	d elete	조건에 맞는 줄을 지운다. 예: '/^#/d'는 #으로 시작하는 줄 삭제.

상황별 예제

01 파일을 직접 고치기 (macOS)

```
% sed -i '' 's/http:/https:/g' index.html
```

macOS에서는 -i 뒤에 빈 따옴표 "가 꼭 필요하다. 리눅스와 다른 점이라 자주 막힌다.

02 백업을 남기며 고치기

```
% sed -i .bak 's/foo/bar/g' config.txt
```

config.txt.bak에 원본이 남는다.

03 주석 줄 지우기

```
% sed '/^#/d' config.txt
```

#으로 시작하는 줄을 빼고 출력한다.

04 10~20번째 줄만 보기

```
% sed -n '10,20p' app.log
```

긴 로그에서 원하는 구간만 잘라 본다.

05 여러 파일 한 번에

```
% sed -i '' 's/2025/2026/g' *.md
```

모든 .md 파일의 연도를 한꺼번에 고친다.

비하인드 스토리

s/틀린말/맞는말/ 문법은 개발자들의 말버릇이 되었다. 채팅에서 오타를 내면 다음 메시지로 "s/회으/회의/"라고만 보내 정정하는 문화가 있을 정도다. 두 글자로 말하는 사람들다운 농담이다.

꿀팁

처음에는 -i 없이 실행해 결과를 눈으로 확인하고, 괜찮으면 그때 -i를 붙이자.

xargs

텍스트 가공

extended arguments

한마디로 앞 명령의 결과를 뒤 명령의 재료로 바꿔준다.

형식 명령1 | xargs 명령2

```
~ % find . -name "*.tmp" | xargs rm
→ 찾아낸 .tmp 파일을 rm에 한꺼번에 넘겨 삭제
```

유래

extended arguments(늘려 주는 인자)로 풀이한다. 파이프(|)가 넘겨주는 글자
문치를 잘라서, 뒤 명령의 인자(명령 뒤에 붙는 재료)로 바꿔 붙여 준다.

주요 옵션

옵션	원래 단어	뜻
-I {}	<u>I</u> nsert (repl <u>a</u> ce)	앞에서 받은 값을 {} 자리에 하나씩 넣어, 명령을 여러 번 실행한다.
-n	max <u>n</u> umber	한 번에 몇 개씩 넘길지 정한다. -n 1이면 하나씩.
-0	null (<u>0</u>)	파일 이름에 띄어쓰기가 있어도 안전하게 나눈다. find -print0과 함께 쓴다.
-p	<u>p</u> rompt	실행하기 전에 매번 "실행할까요?"라고 묻는다.

01 공백 있는 파일도 안전하게

```
% find . -name "*.log" -print0 | xargs -0 rm
```

"my file.log" 같은 이름도 한 덩어리로 처리된다.

02 하나씩 확인하며

```
% ls *.jpg | xargs -p rm
```

지우기 전에 매번 ?...라고 묻는다.

03 목록의 주소를 전부 다운로드

```
% cat urls.txt | xargs -I {} curl -O {}
```

{ } 자리에 주소가 하나씩 들어가 차례로 실행된다.

04 파일마다 줄 수

```
% ls *.md | xargs wc -l
```

파일별 줄 수와 합계가 함께 나온다.

기억법

파이프(|)는 "내용"을 넘기고, xargs는 그 내용을 "명령 뒤에 붙는 재료"로 바꾼다.
rm처럼 입력을 읽지 않고 이름만 받는 명령에 결과를 넘길 때 필요하다.

05 빠른 참조 · 텍스트 가공하기

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>wc -l 파일</code>	줄 수 세기
<code>ls wc -l</code>	파일 개수
<code>sort -n 파일</code>	숫자 순 정렬
<code>sort uniq -c sort -rn</code>	많이 나온 순위
<code>sed 's/a/b/g' 파일</code>	바꾼 결과 미리보기
<code>sed -i '' 's/a/b/g' 파일</code>	파일 직접 수정 (macOS)
<code>find ... -print0 xargs -0 명령</code>	찾은 파일에 한꺼번에 명령

누가 읽고 쓰고 실행할 수 있는지, 누구의 파일인지, 언제 관리자 권한을 빌릴지 정한다.

chmod

권한

change mode

한마디로 **누가 읽고 쓰고 실행할지 정한다.**

형식 **chmod** 숫자|기호 파일명

```
~ % chmod +x deploy.sh
→ 실행 권한 추가, ./deploy.sh 로 실행 가능

~ % chmod 755 deploy.sh
→ 소유자 rwx, 그룹·기타 r-x
```

권한 숫자표

- | | |
|---|---|
| 7 | rwx · 열어 보고, 고치고, 실행까지 모두 할 수 있다. (4+2+1) |
| 6 | rw- · 열어 보고 고칠 수 있지만 실행은 못 한다. (4+2) |
| 5 | r-x · 열어 보고 실행할 수 있지만 고치지는 못 한다. (4+1) |
| 4 | r-- · 열어 보기만 할 수 있다. (4) |
| 0 | --- · 아무것도 할 수 없다. |

기호의 뜻

옵션	원래 단어	뜻
u / g / o / a	u ser · g roup · o thers · a ll	권한을 바꿀 대상. u는 파일 주인, g는 같은 그룹 사람들, o는 그 밖의 모든 사람, a는 셋 모두.
r / w / x	r ead · w rite · e xecute	바꿀 권한. r은 읽기(열어 보기), w는 쓰기(고치기), x는 실행(프로그램처럼 돌리기).
+ / - / =	add · remove · set	+는 권한을 더하고, -는 빼고, =는 정확히 그것만 남긴다. 예: u+x, g-w, o=r
-R	R ecursive	폴더 안의 모든 파일과 하위 폴더에 같은 권한을 한꺼번에 적용한다.

상황별 예제

01 스크립트를 실행 가능하게

```
% chmod +x build.sh
```

이제 ./build.sh 로 바로 실행할 수 있다.

02 SSH 키를 나만 읽게

```
% chmod 600 ~/.ssh/id_ed25519
```

키가 남에게 열려 있으면 ssh가 사용을 거부한다. 600은 나만 읽고 쓰기.

03 웹 폴더 권한 일괄 정리

```
% chmod -R 755 public/
```

나는 모든 권한, 나머지는 읽기와 실행만.

04 그룹의 쓰기 권한만 빼기

```
% chmod g-w shared.txt
```

그룹(g)에서 쓰기(w)를 뺀다(-). 나머지는 그대로.

05 바뀌었는지 확인

```
% ls -l build.sh
```

-rwxr-xr-x 처럼 x가 보이면 실행 권한이 생긴 것이다.

기억법

r=4, w=2, x=1은 2진수 자리값(100, 010, 001)과 같다. 원하는 권한을 더하면 그게 곧 숫자다: 읽기+실행만 원하면 4+1=5.

꿀팁

chmod 777(모두에게 모든 권한)은 편하지만 보안 위험이 크다. 꼭 필요한 최소 권한만 주는 습관을 들이자.

chown

권한

change owner

한마디로 **파일의 주인을 바꾼다.**

형식 **chown** 사용자[:그룹] 파일명

```
~ % sudo chown me:staff data.txt
~ % ls -l data.txt
-rw-r--r-- 1 me staff 812 ... data.txt
```

유래

change owner. chmod가 "무엇을 할 수 있나"를 정한다면, chown은 "누구의 것인가"를 정한다. 남의 파일 주인을 바꾸는 일이라 대부분 sudo와 함께 쓴다.

주요 옵션

옵션	원래 단어	뜻
-R	<u>R</u> ecursive	폴더 안의 모든 파일과 하위 폴더의 주인을 한꺼번에 바꾼다.
-v	<u>v</u> erbose	주인이 바뀐 파일 이름을 보여준다.
-h	symlink (관례상 h)	바로가기가 가리키는 원본 말고, 바로가기 파일 자체의 주인을 바꾼다.

상황별 예제

01 root 것이 된 파일 되찾기

```
% sudo chown $(whoami) report.txt
```

\$(whoami)는 내 사용자 이름으로 바뀐다.

02 npm 권한 오류 해결

```
% sudo chown -R $(whoami) ~/.npm
```

sudo npm으로 설치하다 꼬인 캐시 폴더의 주인을 나로 되돌린다.

03 그룹까지 함께

```
% sudo chown me:staff shared.txt
```

콜론(:) 뒤가 그룹이다.

기억법

chmod는 mode(권한), chown은 owner(주인). 둘 다 change로 시작하는 형제 명령이다.

꿀팁

sudo로 만든 파일은 주인이 root가 되어 나중에 평범하게 수정할 수 없다.
"Permission denied"가 뜨면 ls -l로 주인부터 확인하자.

sudo

권한

superuser do

한마디로 이 명령 하나만 관리자 권한으로 실행한다.

형식 **sudo** 명령어

```
~ % sudo softwareupdate -l
Password:
→ 맥 로그인 비밀번호 입력 (화면에 안 보이는 게 정상)
```

유래

superuser do, "관리자로서 실행하라". 1980년 뉴욕주립대 버펄로에서 처음 만들어졌다. 관리자 계정으로 아예 로그인하지 않고 필요한 순간에만 권한을 빌리게 해 사고를 줄이는 것이 목적이다.

주요 옵션

옵션	원래 단어	뜻
-i	<u>i</u> nteractive login	명령 하나가 아니라 아예 관리자 상태로 들어간다. 프롬프트가 #으로 바뀌고, exit로 나온다.
-k	<u>k</u> ill timestamp	비밀번호 기억을 지워, 다음 sudo 때 다시 묻게 한다.
-u	<u>u</u> ser	관리자 대신 지정한 다른 사용자로 실행한다.

상황별 예제

01 방금 명령을 관리자로 다시

```
% sudo !!
```

!!는 직전 명령. Permission denied가 떴을 때 처음부터 다시 칠 필요가 없다.

02 시스템 파일 수정

```
% sudo nano /etc/hosts
```

특정 사이트 주소를 직접 지정할 때 쓰는 파일이다.

03 DNS 캐시 비우기 (macOS)

```
% sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

사이트 주소가 바뀌었는데 예전 곳으로 접속될 때 쓴다.

04 권한 빌린 상태 끝내기

```
% sudo -k
```

잠시 동안 비밀번호를 다시 묻지 않는 상태를 바로 끝낸다.

비하인드 스토리

유명한 웹툰 xkcd에는 "샌드위치 만들어 줘"라는 말에 "네가 만들어"라고 거절당하던 사람이 "sudo 샌드위치 만들어 줘"라고 하자 "알겠어"라는 답을 듣는 장면이 있다. 이후 sudo는 개발자들 사이에서 "꼭 줌"이라는 뜻의 농담으로도 쓰인다.

꿀팁

인터넷에서 복사한 sudo 명령은 컴퓨터 전체를 바꿀 수 있다. 붙여넣기 전에 무슨 뜻인지 반드시 확인하자.

06 빠른 참조 · 권한과 관리자

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>chmod +x 파일</code>	실행할 수 있게
<code>chmod 600 파일</code>	나만 읽고 쓰기
<code>chmod -R 755 폴더</code>	폴더 전체 권한 정리
<code>sudo chown \$(whoami) 파일</code>	주인을 나로 되돌리기
<code>sudo 명령</code>	관리자 권한으로 한 번 실행
<code>sudo !!</code>	직전 명령을 관리자로 다시

여러 파일을 하나로 묶거나 다시 푸는 두 가지 방법. 맥·리눅스끼리는 tar, 윈도우와는 zip.

tar

압축

tape archive

한마디로 여러 파일을 하나로 묶고 푼다.

형식 **tar** 옵션 파일명.tar.gz 대상

```
~ % tar -czvf backup.tar.gz src/
→ src 폴더를 backup.tar.gz 하나로 압축
~ % tar -xzvf backup.tar.gz
→ 현재 위치에 압축 풀기
```


주요 옵션

옵션	원래 단어	뜻
-c	<u>c</u> reate	새 압축 파일을 만든다.
-x	e <u>x</u> tract	압축 파일을 풀어 원래 파일들로 꺼낸다.
-t	<u>t</u> able of contents	폴지 않고 안에 든 파일 목록만 보여준다.
-z	g <u>z</u> ip	gzip 방식으로 압축하거나 푼다. 이름이 .tar.gz인 파일을 다룰 때 필요하다.
-v	<u>v</u> erbose	처리 중인 파일 이름을 하나씩 보여준다.
-f	<u>f</u> ile	바로 뒤에 압축 파일 이름이 온다는 표시. 그래서 항상 옵션의 맨 끝에 둔다.
-C	<u>C</u> hange directory	압축을 풀 폴더를 지정한다. 없으면 지금 있는 폴더에 풀린다.

유래

tape + archive. 1970년대에 여러 파일을 자기 테이프(tape)에 순서대로 이어 기록해 보관(archive)하던 방식에서 이름이 그대로 남았다. 여러 파일이 하나로 뭉쳐진 결과물을 "타볼(tarball)"이라 부르는 것도 여기서 나온 은어다.

상황별 예제

01 풀기 전에 안을 들여다보기

```
% tar -tzvf backup.tar.gz
```

압축을 풀지 않고 들어 있는 파일 목록만 본다.

02 원하는 폴더에 풀기

```
% tar -xzf backup.tar.gz -C ~/restore
```

-C로 지정한 폴더 안에 풀린다. 폴더는 미리 있어야 한다.

03 여러 대상을 하나로

```
% tar -czvf site.tar.gz index.html css/ js/
```

파일과 폴더를 섞어 한 파일로 묶는다.

04 무거운 폴더는 빼고

```
% tar -czvf project.tar.gz --exclude=node_modules project/
```

다시 받을 수 있는 node_modules는 제외해 용량을 줄인다.

05 날짜가 붙은 백업 파일

```
% tar -czf docs-$(date +%Y%m%d).tar.gz docs/
```

docs-20260930.tar.gz 처럼 날짜가 이름에 들어간다.

꿀팁

압축을 풀지 않고 안에 뭐가 들었는지만 보고 싶다면 tar -tzvf 파일명 을 쓰면 된다.

zip

압축

zip (지퍼) · unzip

한마디로 윈도우와도 호환되는 zip 파일로 묶고 푼다.

형식 `zip -r 이름.zip 대상 / unzip 이름.zip`

```
~ % zip -r photos.zip photos/
  adding: photos/a.jpg (deflated 3%)
~ % unzip photos.zip -d restore/
→ restore 폴더에 압축 해제
```

유래

지퍼(zip)처럼 빠르게 잠근다는 뜻. 1989년 필 카츠가 만든 PKZIP에서 시작된 형식으로, 지금은 운영체제를 가리지 않는 가장 흔한 압축 형식이 되었다.

주요 옵션

옵션	원래 단어	뜻
-r	<u>r</u> ecursive	폴더를 안의 내용까지 통째로 압축한다. 폴더를 압축할 때 필수다.
-e	<u>e</u> ncrypt	압축 파일에 비밀번호를 건다. 확인을 위해 두 번 입력한다.
-x	<u>e</u> xclude	압축에서 뺄 파일을 지정한다. 예: -x "*.DS_Store"
-q	<u>q</u> uiet	압축하는 파일 목록을 화면에 보여주지 않는다.
unzip -l	<u>l</u> ist	폴지 않고 안에 든 파일 목록만 보여준다.
unzip -d	<u>d</u> irectory	압축을 풀 폴더를 지정한다.

01 비밀번호 걸어 보내기

```
% zip -er secret.zip contracts/
```

두 번 입력한 비밀번호가 있어야 열 수 있다.

02 맥 찌꺼기 빼고 압축

```
% zip -r site.zip site -x "*.DS_Store"
```

윈도우 사용자에게 보낼 때 깔끔하다.

03 열기 전에 내용 확인

```
% unzip -l archive.zip
```

어떤 파일이 들었는지 목록만 본다.

꿀팁

Finder의 "압축하기"로 만든 zip에는 __MACOSX라는 숨은 폴더가 섞여 윈도우 사용자가 당황하곤 한다. 터미널의 zip을 쓰면 이 문제가 없다.

기억법

맥·리눅스끼리는 tar.gz, 윈도우 사용자에게 보낼 때는 zip. 받는 사람을 먼저 생각하면 고르기 쉽다.

07 빠른 참조 · 압축과 보관

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>tar -czvf 이름.tar.gz 대상</code>	tar로 압축
<code>tar -xzvf 이름.tar.gz</code>	tar 풀기
<code>tar -tzvf 이름.tar.gz</code>	폴지 않고 목록만
<code>zip -r 이름.zip 대상</code>	zip으로 압축
<code>zip -er 이름.zip 대상</code>	비밀번호 걸어 압축
<code>unzip 이름.zip -d 폴더</code>	zip 풀기

연결을 확인하고, 인터넷에서 데이터를 가져오고, 다른 컴퓨터에 안전하게 접속해 파일을 주고받는다.

ping

네트워크

ping (소나의 핑 소리)

한마디로 **상대 컴퓨터에 신호를 보내 응답을 확인한다.**

형식 **ping** 주소

```
~ % ping -c 3 google.com
64 bytes from 142.250.206.46: icmp_seq=0 ttl=117 time=31.2 ms
3 packets transmitted, 3 packets received, 0.0% packet loss
```

유래

잠수함 소나가 음파를 쏘고 되돌아오는 "핑" 소리에서 이름을 땀다. 1983년 마이크 무스가 하룻밤 사이에 만들었다. 흔히 Packet InterNet Groper의 약자로 소개되지만, 이는 나중에 끼워 맞춘 풀이다.

주요 옵션

옵션	원래 단어	뜻
-c	count	신호를 몇 번 보낼지 정한다. 없으면 Control+C를 누를 때까지 계속 보낸다.
-i	interval	신호를 보내는 간격(초)을 정한다. 기본은 1초.
-t	timeout	몇 초가 지나면 자동으로 멈출지 정한다.

상황별 예제

01 인터넷이 되나

```
% ping -c 3 8.8.8.8
```

응답이 오면 인터넷 연결 자체는 살아 있다.

02 인터넷 문제인가 주소 문제인가

```
% ping -c 3 google.com
```

8.8.8.8은 되는데 이게 안 되면 주소를 찾아주는 DNS 문제다.

03 공유기까지는 되나

```
% ping 192.168.0.1
```

집 안 연결부터 점검한다. Control+C로 멈추면 통계가 나온다.

비하인드 스토리

time=31.2 ms는 신호가 갔다가 돌아오는 데 걸린 시간이다. 게이머들이 말하는 "핑이 튼다"의 핑이 바로 이 숫자다.

curl

네트워크

client URL

한마디로 인터넷 주소와 데이터를 주고받는다.

형식 **curl** [옵션] URL



```
~ % curl -O https://example.com/file.zip
```

→ 원본 파일명 그대로 다운로드

```
~ % curl -I https://example.com
```

```
HTTP/2 200
```

```
content-type: text/html
```


주요 옵션

옵션	원래 단어	뜻
-o	<u>o</u> utput	받은 내용을 내가 정한 이름의 파일로 저장한다. 예: -o logo.png
-O	remote-name <u>O</u> utput	받은 파일을 서버에 있던 이름 그대로 저장한다.
-L	<u>L</u> ocation	주소가 다른 곳으로 옮겨졌으면 따라가서 받는다. 없으면 빈 파일을 받기도 한다.
-I	head (<u>I</u> nfo)	내용은 빼고, 서버의 응답 정보(상태 번호, 파일 형식 등)만 본다.
-s	<u>s</u> ilent	진행률 같은 표시를 숨기고 결과만 보여준다.
-X	request (관례상 X)	요청 방식을 정한다. 가져오기는 GET(기본값), 보내기는 POST.
-H	<u>H</u> header	요청에 추가 정보를 붙인다. 예: 데이터 형식, 인증 키.
-d	<u>d</u> ata	서버로 보낼 데이터를 적는다. 쓰면 자동으로 POST 방식이 된다.

유래

"Client URL"의 줄임말. 1996년 스웨덴 개발자 다니엘 스텐베리가 만든 httpget이 urlget을 거쳐 curl로 이름을 바꿨다.

상황별 예제

01 이름을 바꿔 저장

```
% curl -o logo.png https://example.com/image.png
```

소문자 o 뒤에 원하는 파일 이름을 적는다.

02 리다이렉트되는 링크 받기

```
% curl -LO https://example.com/latest.zip
```

GitHub 릴리스처럼 주소가 한 번 옮겨지는 링크는 -L이 필요하다.

03 터미널에서 날씨 보기

```
% curl wttr.in/Seoul
```

서울 날씨가 글자 그림으로 나온다. curl로 할 수 있는 가벼운 장난.

04 사이트가 살아 있나

```
% curl -s -o /dev/null -w "%{http_code}" https://example.com
```

200이면 정상, 404는 없음, 500번대는 서버 오류다.

05 API에 데이터 보내기

```
% curl -X POST -H "Content-Type: application/json" -d '{"name": "kim"}' https://api.example.com/users
```

JSON 데이터를 POST 방식으로 서버에 보낸다.

꿀팁

API 키 같은 민감정보를 -H나 -d에 직접 적으면 터미널 히스토리에 평문으로 남는다. 환경변수나 별도 설정 파일로 분리하는 편이 안전하다.

wget

네트워크

web get

한마디로 인터넷에서 파일을 내려받는다.

형식 **wget URL**

```
% wget https://example.com/file.zip
→ 현재 위치에 file.zip으로 자동 저장
```

주요 옵션

옵션	원래 단어	뜻
-O	<u>O</u> utput document	저장할 파일 이름을 직접 정한다.
-c	<u>c</u> ontinue	다운로드가 끊겼을 때 처음부터가 아니라 받던 곳부터 이어 받는다.
-i	<u>i</u> nput file	주소가 한 줄에 하나씩 적힌 파일을 주면, 목록의 파일을 차례로 모두 받는다.
-r	<u>r</u> ecursive	페이지 안의 링크를 따라가며 연결된 파일까지 받는다.

curl과 무엇이 다른가

wget	다운로드 특화, 재귀 다운로드(-r) 지원, 이어받기 강함
curl	API 호출 특화, 헤더·메소드 세밀 제어

유래

web + get. GNU 프로젝트로 1996년 등장했고, 원래 후보 이름은 "geturl"이었지만 상표 문제로 wget이 되었다. macOS엔 기본 내장돼 있지 않아 보통 brew install wget으로 따로 설치한다.

상황별 예제

01 설치부터

```
% brew install wget
```

macOS에는 기본으로 없으니 Homebrew로 설치한다.

02 끊긴 다운로드 이어받기

```
% wget -c https://example.com/big.iso
```

받던 곳부터 이어서 받는다. 큰 파일에 유용하다.

03 이름을 정해서 저장

```
% wget -O data.csv https://example.com/export
```

대문자 O 뒤에 저장할 이름을 적는다.

04 목록에 있는 파일 전부

```
% wget -i urls.txt
```

한 줄에 하나씩 적힌 주소를 차례로 모두 받는다.

ssh

네트워크

secure shell

한마디로 다른 컴퓨터에 안전하게 접속한다.

형식 **ssh** 사용자명@호스트주소

```
~ % ssh user@example.com
→ 비밀번호 또는 키 인증 후 원격 셸 접속
~ % ssh -i ~/.ssh/id_ed25519 user@example.com
→ 지정한 키 파일로 접속
```

주요 옵션

옵션	원래 단어	뜻
-p	p ort	접속할 포트(문) 번호를 정한다. 적지 않으면 22번.
-i	i dentity file	비밀번호 대신 쓸 개인 키 파일을 지정한다.
-v	v erbose	접속 과정을 자세히 보여준다. 접속이 안 될 때 원인을 찾는 용도.
-L	L ocal forward	서버의 특정 포트를 내 맥의 포트로 이어준다. 예: 서버의 데이터베이스를 내 맥에서 열기.

자주 같이 쓰는 도구

scp(파일 복사), ssh-keygen(키 생성), ~/.ssh/config(자주 쓰는 서버 별칭 등록)

01 포트가 다른 서버

```
% ssh -p 2222 user@example.com
```

관리자가 기본 22번 대신 다른 포트를 열어둔 경우다.

02 접속하지 않고 명령만

```
% ssh user@example.com "df -h"
```

서버의 디스크 상태만 받아오고 바로 연결을 끝낸다.

03 내 키 만들기

```
% ssh-keygen -t ed25519 -C "me@example.com"
```

~/.ssh/에 개인키와 공개키(.pub)가 생긴다.

04 비밀번호 없이 접속하게

```
% ssh-copy-id user@example.com
```

내 공개키를 서버에 등록한다. 다음부터는 키로 자동 인증된다.

05 접속 끝내기

```
% exit
```

원래 내 맥의 터미널로 돌아온다. Control+D도 같다.

비하인드 스토리

1995년 핀란드 헬싱키공과대학의 타투 월로넨은 자신이 속한 대학 네트워크에서 비밀번호를 가로채는 스니핑 공격을 직접 당했다. 당시 널리 쓰이던 rlogin과 telnet은 비밀번호를 암호화 없이 그대로 전송하고 있었기 때문. 이 경험이 계기가 되어 그는 암호화된 원격 접속 프로그램 ssh를 직접 만들었고, 오늘날까지 원격 접속의 표준이 되었다.

scp

네트워크

secure copy

한마디로 다른 컴퓨터와 파일을 주고받는다.

형식 scp 원본 대상

```
~ % scp file.txt user@example.com:~/backup/
→ 로컬 → 원격 업로드
~ % scp -r src/ user@example.com:~/app/
→ 폴더 전체 복사(재귀)
```

주요 옵션

옵션	원래 단어	뜻
-r	recursive	폴더를 안의 내용까지 통째로 복사한다.
-P	Port	포트 번호를 정한다. ssh와 달리 대문자 P다. 소문자 -p는 다른 뜻.
-p	preserve	원본의 수정 날짜와 권한을 그대로 유지한다.
-i	identity file	비밀번호 대신 쓸 개인 키 파일을 지정한다.

유래

secure copy. ssh 위에서 동작하며, 비밀번호를 암호화 없이 주고받던 예전의 rcp(remote copy)를 대체하기 위해 등장했다.

01 서버 파일을 내 맥으로

```
% scp user@example.com:~/log.txt ./
```

콜론(:) 뒤가 서버 쪽 경로, ./는 지금 폴더다.

02 사진 여러 장 올리기

```
% scp *.jpg user@example.com:~/photos/
```

현재 폴더의 jpg가 모두 서버로 올라간다.

03 포트가 다를 때

```
% scp -P 2222 file.txt user@example.com:~/
```

ssh와 달리 대문자 P다.

04 별칭으로 짧게

```
% scp file.txt myserver:~/
```

~/ssh/config에 등록된 별칭을 그대로 쓸 수 있다.

꿀팁

ssh는 소문자 -p, scp는 대문자 -P로 포트를 지정한다. 실무에서 가장 자주 헛갈리는 옵션 중 하나다.

rsync

네트워크

remote sync

한마디로 **바뀐 부분만 골라 동기화한다.**

형식 **rsync** 옵션 원본 대상

```
~ % rsync -avP src/ backup/
sending incremental file list
App.tsx 12,048 100% 4.21MB/s 0:00:00
~ % rsync -avn src/ backup/
→ dry-run, 실제 복사 없이 무엇이 바뀔지만 미리 확인
```

주요 옵션

옵션	원래 단어	뜻
-a	<u>a</u> rchive	폴더를 통째로, 권한·날짜·링크까지 원본 그대로 복사한다. 백업할 때 기본으로 쓴다.
-v	<u>v</u> erbose	복사하는 파일 이름을 보여준다.
-z	<u>z</u> ip (compress)	네트워크로 보낼 때 압축해서 보내 시간을 줄인다.
-P	<u>P</u> artial + <u>P</u> rogress	진행률을 보여주고, 끊기면 보내던 곳부터 이어서 보낸다.
-n	<u>n</u> o action (dry-run)	실제로 복사하지 않고 무엇이 복사·삭제될 지 목록만 보여주는 예행연습.
--delete	delete	원본에서 지운 파일은 대상에서도 지워 두 폴더를 똑같이 맞춘다.
--exclude	exclude	복사하지 않을 파일이나 폴더를 지정한다. 예: node_modules

cp/scp와 다른 점

파일 전체를 다시 보내지 않고 달라진 부분(delta)만 찾아 전송한다. 그래서 큰 폴더를 반복 동기화할 때 훨씬 빠르다.

상황별 예제

01 외장하드로 백업

```
% rsync -av ~/Documents/ /Volumes/Backup/Documents/
```

두 번째부터는 바뀐 파일만 복사돼 순식간에 끝난다.

02 무거운 폴더는 빼고

```
% rsync -av --exclude='node_modules' project/ backup/
```

제외할 이름을 여러 번 지정할 수도 있다.

03 웹서버에 배포

```
% rsync -avz dist/ user@example.com:/var/www/html/
```

빌드 결과물을 압축 전송해 서버에 올린다.

04 실행 전 리허설

```
% rsync -avn --delete src/ mirror/
```

무엇이 복사되고 지워질지 목록만 보여준다. 확인 후 n을 뺀다.

비하인드 스토리

1996년 호주 개발자 앤드루 트리젤(삼바 SMB 서버 개발자로도 유명하다)은 박사 논문에서 "두 파일의 차이 나는 조각만 찾아 전송하는" 알고리즘을 제안했다. 이 알고리즘을 그대로 구현한 것이 rsync이며, 지금도 백업 도구 대부분이 이 방식을 빌려 쓰고 있다.

꿀팁

--delete는 대상 쪽 파일을 지울 수 있어 되돌릴 수 없다. 실행 전 -n(dry-run)으로 먼저 확인하는 습관을 들이자.

08 빠른 참조 · 네트워크와 원격 접속

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>ping -c 3 8.8.8.8</code>	인터넷 연결 확인
<code>curl -LO 주소</code>	파일 다운로드
<code>curl -I 주소</code>	응답 헤더만
<code>wget -c 주소</code>	끊긴 다운로드 이어받기
<code>ssh user@host</code>	서버 접속
<code>ssh-keygen -t ed25519</code>	SSH 키 만들기
<code>scp 파일 user@host:경로</code>	서버로 파일 보내기
<code>rsync -av 원본/ 대상/</code>	바뀐 파일만 동기화

지금 실행 중인 프로그램을 확인하고, 무엇을 붙잡고 있는지 보고, 필요하다면 멈춘다.

ps

프로세스

process status

한마디로 **지금 돌고 있는 프로그램을 보여준다.**

형식 **ps [옵션]**

```
~ % ps aux | grep node
me 12345 0.3 1.2 node server.js
```

주요 옵션

옵션	원래 단어	뜻
a	<u>a</u> ll users	나뿐 아니라 다른 사용자가 실행한 프로그램도 보여준다.
u	<u>u</u> ser format	누가 실행했는지, CPU·메모리를 얼마나 쓰는지 함께 보여준다.
x	<u>e</u> xtended	터미널 창과 상관없이 뒤에서 도는 앱과 서비스까지 보여준다.
-r	<u>r</u> esource (CPU)	CPU를 많이 쓰는 프로그램이 위로 오게 정렬한다. (macOS)
-m	<u>m</u> emory	메모리를 많이 쓰는 프로그램이 위로 오게 정렬한다. (macOS)

핵심 컬럼

PID 프로세스 고유 번호(종료할 때 필요), %CPU·%MEM 사용률, COMMAND 실행 명령어

상황별 예제

01 지금 이 창의 프로세스

```
% ps
```

옵션 없이 쓰면 현재 터미널에서 띄운 것만 보인다.

02 CPU를 가장 많이 쓰는 5개

```
% ps aux -r | head -6
```

맥이 느려졌을 때 범인을 찾는 가장 빠른 방법 (첫 줄은 제목).

03 메모리 대식이 찾기

```
% ps aux -m | head -6
```

메모리 사용량 순으로 정렬한다.

04 실시간으로 지켜보기

```
% top -o cpu
```

계속 갱신되는 화면. q로 빠져나온다.

top

프로세스

top (맨 위)

한마디로 **자원을 가장 많이 쓰는 프로그램을 실시간으로 보여준다.**

형식 **top [-o 정렬기준]**

```
~ % top -o cpu
PID      COMMAND          %CPU  MEM
4821     node              87.3   412M
612      WindowServer     12.1   256M
→ 계속 갱신, q로 종료
```

유래

자원을 가장 많이 쓰는, 순위의 맨 위(top) 프로그램들을 보여준다는 뜻. 1984년 윌리엄 르페브르가 만들었다. macOS의 활동 모니터 앱과 같은 정보를 터미널에서 본다.

주요 옵션

옵션	원래 단어	뜻
-o	<u>o</u> rder	무엇을 기준으로 줄 세울지 정한다. cpu는 CPU 사용량, mem은 메모리 사용량.
-n	<u>n</u> umber	화면에 보여줄 프로그램 개수를 정한다.
-l	<u>l</u> ogging samples	계속 갱신하지 않고, 정한 횟수만큼만 결과를 찍고 끝낸다. (macOS)
q	<u>q</u> uit	top 화면을 닫고 터미널로 돌아온다.

01 메모리 먹는 순서

```
% top -o mem
```

메모리가 부족하다는 경고가 뜰 때 범인을 찾는다.

02 상위 10개만

```
% top -o cpu -n 10
```

화면이 복잡할 때 10개만 본다.

03 한 번만 찍고 끝

```
% top -l 1 | head -12
```

실시간 화면 대신 결과를 한 번만 받아 다른 명령에 넘긴다.

꿀팁

더 보기 좋은 버전으로 htop이 있다. brew install htop으로 설치하면 색깔과 막대그래프로 보여준다.

kill

프로세스

신호(signal)를 보내는 명령

한마디로 프로그램에 신호를 보내 멈추게 한다.

형식 `kill [-신호번호] PID`

```
~ % kill 12345
→ 정상 종료 신호(SIGTERM) 전송, 프로세스가 정리 후 종료
~ % kill -9 12345
→ 강제 종료(SIGKILL), 즉시 종료
```

신호와 옵션

옵션	원래 단어	뜻
-15	SIG <u>TERM</u> · terminate	프로그램에게 "정리하고 끝내 주세요"라고 부탁한다. 숫자를 안 쓰면 이것이다.
-2	SIG <u>INT</u> · interrupt	터미널에서 Control+C를 누른 것과 같다. 하던 일을 중단시킨다.
-9	SIG <u>KILL</u> · kill	프로그램이 거부할 수 없는 강제 종료. 저장하지 않은 내용은 사라진다.
-1	SIG <u>HUP</u> · hang up	원래 뜻은 "연결 끊김". 요즘은 서버 프로그램에 설정을 다시 읽으라고 할 때 쓴다.
-l	<u>l</u> ist	보낼 수 있는 신호의 이름을 모두 보여준다.

이름으로 종료

`killall node` 이름이 node인 프로세스 전부 종료, `pkill -f "server"` 명령어에 server가 포함된 프로세스 종료

상황별 예제

01 3000번 포트를 켜 버린

```
% lsof -i :3000
```

개발 서버가 안 켜질 때, 그 포트를 붙잡은 프로세스의 PID를 알려준다.

02 찾자마자 종료

```
% kill $(lsof -t -i :3000)
```

-t는 PID만 출력. 그 번호를 바로 kill에 넘긴다.

03 응답 없는 앱 닫기

```
% killall Safari
```

이름이 같은 프로세스를 모두 종료한다.

04 명령어 일부로 찾아 종료

```
% pkill -f "node server.js"
```

-f는 실행 명령 전체 문자열에서 찾는다.

05 보낼 수 있는 신호 목록

```
% kill -l
```

HUP INT QUIT ... 같은 신호 이름이 나열된다.

기억법

이름은 "죽이다"지만 사실은 "신호를 보낸다"가 더 정확하다. -9만 진짜로 무조건 죽이고, 나머지는 프로그램이 받아서 알아서 처리하도록 부탁하는 신호다.

비하인드 스토리

SIGHUP의 HUP은 "Hang Up", 옛 전화선을 끊는다는 뜻이다. 원래는 터미널과의 연결이 끊겼을 때 커널이 자동으로 보내던 신호였는데, 오늘날 많은 서버 프로그램들은 이 신호를 "설정 파일을 다시 읽어라"는 의미로 재활용해서 쓰고 있다.

lsnf

프로세스

list open files

한마디로 어떤 프로그램이 무엇을 붙잡고 있는지 보여준다.

형식 **lsnf** [옵션]

```
~ % lsnf -i :3000
COMMAND  PID  USER  FD  TYPE  NAME
node     4821  me    23u  IPv6  *:3000 (LISTEN)
```

유래

list open files, 열려 있는 파일 목록. 유닉스에는 "모든 것은 파일이다"라는 철학이 있어서 네트워크 연결까지 파일처럼 다룬다. 그래서 파일 목록 명령으로 포트까지 찾을 수 있다.

주요 옵션

옵션	원래 단어	뜻
-i	<u>i</u> nternet	네트워크 연결만 본다. -i :3000처럼 포트 번호를 붙일 수 있다.
-t	<u>t</u> erse	다른 정보 없이 프로그램 번호(PID)만 보여준다. kill과 함께 쓰기 좋다.
-p	<u>p</u> rocess	지정한 번호(PID)의 프로그램이 열고 있는 파일을 보여준다.
+D	<u>D</u> irectory	지정한 폴더 안에서 열려 있는 파일을 모두 찾는다.

01 포트를 붙잡은 범인 종료

```
% kill $(lsof -t -i :3000)
```

"port already in use" 오류가 날 때 한 줄로 해결한다.

02 외장하드가 추출되지 않을 때

```
% lsof +D /Volumes/USB
```

그 디스크의 파일을 열고 있는 프로그램이 나온다. 그 프로그램을 닫으면 추출된다.

03 열려 있는 서버 포트 전부

```
% lsof -i -P | grep LISTEN
```

내 맥에서 접속을 기다리는 프로그램 목록이다.

비하인드 스토리

"모든 것은 파일이다(Everything is a file)"는 유닉스의 가장 유명한 설계 원칙이다. 키보드, 화면, 네트워크 연결까지 파일처럼 읽고 쓰게 만들어서, 적은 수의 두 글자 명령으로 거의 모든 것을 다룰 수 있게 되었다.

09 빠른 참조 · 프로세스 관리

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>ps aux grep 이름</code>	특정 프로세스 찾기
<code>top -o cpu</code>	CPU 순 실시간 보기
<code>top -o mem</code>	메모리 순
<code>kill PID</code>	정상 종료 요청
<code>kill -9 PID</code>	강제 종료
<code>lsof -i :3000</code>	포트 쓰는 프로세스
<code>kill \$(lsof -t -i :3000)</code>	포트 붙잡은 프로세스 바로 종료

텍스트가 얼마나 다른지, 디스크를 얼마나 쓰고 있는지 확인하는 명령어.

diff

비교·용량

difference

한마디로 두 파일의 다른 곳을 짚어준다.

형식 `diff 파일1 파일2`

```
~ % diff -u a.txt b.txt
< 강의 3장 초안
---
> 강의 3장 수정본
```

출력 읽는 법

<는 왼쪽(a.txt)에만 있는 줄, >는 오른쪽(b.txt)에만 있는 줄을 뜻한다.

주요 옵션

옵션	원래 단어	뜻
-u	<u>u</u> nified	바뀐 줄의 앞뒤를 함께 보여주고 +(추가), -(삭제)로 표시한다. git과 같은 형식.
-y	side-b <u>y</u> -side	두 파일을 왼쪽·오른쪽에 나란히 놓고 비교한다.
-r	<u>r</u> ecursive	폴더 안의 파일들까지 모두 비교한다.
-q	<u>q</u> uiet (brief)	어디가 다른지는 빼고 "다르다"는 사실만 알려준다.
-w	ignore all <u>w</u> hitespace	띄어쓰기나 들여쓰기만 다른 곳은 같다고 본다.
-i	<u>i</u> gnore case	대문자와 소문자 차이는 무시한다.

상황별 예제

01 좌우로 나란히

```
% diff -y old.txt new.txt
```

두 파일을 양쪽에 놓고 다른 줄에 표시를 붙인다.

02 두 폴더에서 다른 파일만

```
% diff -rq folder1 folder2
```

내용은 생략하고 어떤 파일이 다른지만 알려준다.

03 띄어쓰기 차이는 무시

```
% diff -w a.py b.py
```

들여쓰기나 공백만 다른 줄은 같다고 본다.

04 변경 내역을 파일로

```
% diff -u old.txt new.txt > changes.patch
```

> 는 결과를 파일로 저장. 이 patch 파일로 남에게 수정 사항을 전달한다.

05 두 폴더의 목록 비교

```
% diff <(ls dir1) <(ls dir2)
```

<()는 명령 결과를 임시 파일처럼 넘기는 zsh 문법이다.

비하인드 스토리

1974년 벨 연구소의 더글러스 매킬로이가 만들었다. 두 텍스트에서 "최장 공통 부분열"을 찾는 알고리즘(훗날 헛날-매킬로이 알고리즘으로 불림)을 기반으로 하며, 오늘날 git diff와 patch 명령의 뿌리가 바로 이 프로그램이다.

꿀팁

git으로 관리되는 프로젝트라면 diff 대신 git diff를 쓰는 편이 색상 표시나 변경 이력 면에서 더 편리하다.

du

비교·용량

disk usage

한마디로 **폴더가 차지한 용량을 잴다.**

형식 **du -sh** 폴더명

```
~ % du -sh *
1.2G  node_modules
48M   dist
3.1M  src
```

주요 옵션

옵션	원래 단어	뜻
-s	<u>s</u> ummarize	하위 폴더를 하나하나 나열하지 않고, 폴더 전체의 합계 한 줄만 보여준다.
-h	<u>h</u> uman-readable	크기를 바이트 숫자 대신 K(킬로바이트), M(메가바이트), G(기가바이트)로 보여준다. 예: 1073741824 → 1.0G
-d	<u>d</u> epth	몇 단계 아래 폴더까지 나눠 보여줄지 정한다. -d 1이면 바로 아래 폴더들만.
-a	<u>a</u> ll	폴더뿐 아니라 파일 하나하나의 크기도 보여준다.

df와 구분하기

du는 "이 폴더가 얼마나 썼는지", df는 "디스크 전체가 얼마나 남았는지" — 짝지어 외우면 헷갈리지 않는다.

01 용량 순위 TOP 10

```
% du -sh * | sort -rh | head -10
```

현재 폴더에서 가장 무거운 항목 10개.

02 홈 폴더 한 단계만

```
% du -h -d 1 ~
```

하위 폴더별 합계만 보여준다. 어디가 무거운지 큰 그림을 본다.

03 캐시가 얼마나 쌓였나

```
% du -sh ~/Library/Caches
```

앱들이 쌓아둔 캐시 용량. 수 GB인 경우도 흔하다.

04 node_modules의 무게

```
% du -sh node_modules
```

프로젝트 하나에 수백 MB가 나오기도 한다.

꿀팁

du -sh * | sort -rh 로 용량 큰 항목부터 정렬해서 보면 무엇이 용량을 많이 차지하는지 바로 눈에 띈다.

df

비교·용량

disk free

한마디로 디스크에 남은 공간을 잔다.

형식 **df -h**

```
~ % df -h /
Filesystem      Size  Used Avail Capacity  Mounted on
/dev/disk3s1s1  460Gi  10Gi  240Gi     5%      /
```

옵션과 출력 컬럼

옵션	원래 단어	뜻
-h	<u>h</u> uman-readable	크기를 K·M·G 같은 읽기 쉬운 단위로 보여준다. 1K를 1024바이트로 계산한다.
-H	<u>H</u> uman (SI)	읽기 쉬운 단위로 보여주되 1K를 1000바이트로 계산한다. Finder에 보이는 숫자와 같아진다.
Avail	<u>avail</u> able	앞으로 더 쓸 수 있는 남은 공간.
Capacity	<u>capacity</u> used	전체 용량 중 이미 쓴 비율. 리눅스 서버에서는 같은 칸이 Use%로 표시된다.

du vs df

du -sh 폴더 → 특정 폴더 용량 확인. df -h → 디스크(파티션) 전체 여유공간 확인.

상황별 예제

01 맥 본체 디스크만

```
% df -h /
```

가장 자주 보는 형태. Avail이 남은 공간이다.

02 Finder와 같은 숫자로

```
% df -H /
```

Finder는 1000 단위로 계산한다. 숫자가 다르게 보일 때 이걸 쓴다.

03 외장하드 남은 공간

```
% df -h /Volumes/Backup
```

연결된 외장 디스크는 /Volumes 아래에 있다.

04 지금 폴더가 속한 디스크

```
% df -h .
```

.은 현재 폴더. 이 폴더가 있는 디스크 정보를 보여준다.

10 빠른 참조 · 비교와 용량 확인

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

```
diff -u a b
```

두 파일 차이

```
diff -rq 폴더1 폴더2
```

다른 파일 목록만

```
du -sh * | sort -rh
```

무거운 항목 순위

```
du -h -d 1 ~
```

홈 폴더를 폴더별 용량으로

```
df -h /
```

디스크 남은 공간

```
df -H /
```

Finder와 같은 단위로

11 셸 환경과 도움말

터미널을 내 스타일로 꾸미고, 지난 명령을 되살리고, 모르는 것은 설명서에서 찾는다.

alias

셸 설정

별명

한마디로 긴 명령에 짧은 별명을 붙인다.

형식 `alias 별칭='명령어'`

```
~ % alias ll='ls -lah'
```

```
~ % ll
```

→ `ls -lah`가 실행된 것과 동일한 결과

확인·삭제·영구 적용

`alias`

등록된 목록 전체 보기

`unalias ll`

해당 별칭 삭제

`~/.zshrc`

여기에 적어야 터미널을 새로 열어도 유지됨

유래

특별한 약자 없이 영단어 "별명(alias)"을 그대로 가져온 이름이다.

01 자주 가는 폴더로 순간이동

```
% alias proj='cd ~/Projects'
```

이제 proj만 치면 프로젝트 폴더로 이동한다.

02 삭제 전 확인을 기본으로

```
% alias rm='rm -i'
```

실수 방지용 안전장치. 급할 땐 \rm으로 원래 rm을 쓴다.

03 git 명령 줄이기

```
% alias gs='git status'
```

하루에 수십 번 치는 명령일수록 효과가 크다.

04 이 이름의 정체 확인

```
% type ll
```

ll is an alias for ls -lah 처럼 무엇의 별명인지 알려준다.

05 영구 등록을 한 줄로

```
% echo "alias ll='ls -lah'" >> ~/.zshrc
```

>>는 파일 끝에 덧붙이기. 적용하려면 source ~/.zshrc.

꿀팁

alias로 ls 같은 기존 명령어 이름을 덮어써도, \ls 처럼 백슬래시를 앞에 붙이면 원본 명령어를 그대로 쓸 수 있다.

export

셸 설정

환경변수 내보내기

한마디로 **환경변수를 다른 프로그램에도 넘겨준다.**

형식 **export 변수명=값**

```
~ % export NODE_ENV=production
~ % echo $NODE_ENV
production
```

export 유무 차이

MY_VAR=hello

현재 셸에서만 쓰임

export MY_VAR=hello

이 셸이 실행하는 다른 프로그램에도 전달됨

확인·삭제

env 전체 환경변수 목록, unset 변수명 삭제

상황별 예제

01 내 스크립트 폴더를 PATH에

```
% export PATH="$HOME/bin:$PATH"
```

~/bin 안의 프로그램을 어느 폴더에서든 이름만으로 실행할 수 있다.

02 이 명령에만 잠깐

```
% NODE_ENV=test npm test
```

export 없이 앞에 붙이면 그 명령 한 번에만 적용된다.

03 기본 편집기 정하기

```
% export EDITOR=nano
```

git 커밋 메시지 등을 쓸 때 nano가 열린다.

04 값 확인하기

```
% printenv HOME
```

/Users/me 처럼 변수의 값만 출력한다.

05 변수 지우기

```
% unset NODE_ENV
```

현재 창에서 그 변수가 사라진다.

꿀팁

API 키·비밀번호는 export로 셸 설정 파일에 평문으로 남기지 말고, .env 파일 + .gitignore 조합으로 분리 관리하는 것이 안전하다.

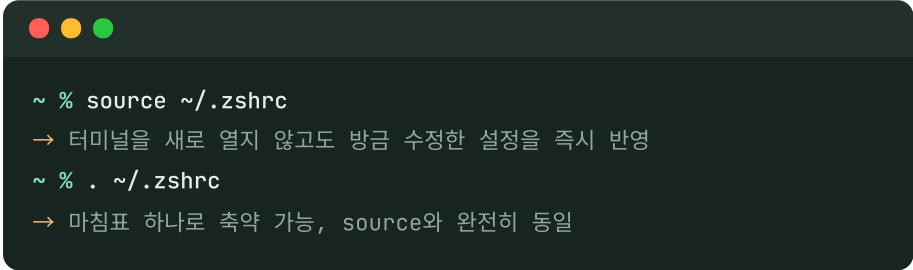
source

셸 설정

원본을 지금 셸에 적용

한마디로 **설정 파일을 지금 창에 바로 적용한다.**

형식 **source** 파일명



```
~ % source ~/.zshrc
→ 터미널을 새로 열지 않고도 방금 수정한 설정을 즉시 반영

~ % . ~/.zshrc
→ 마침표 하나로 축약 가능, source와 완전히 동일
```

왜 필요한가

~/.zshrc에 alias나 export를 추가해도 이미 열려 있는 터미널엔 자동 반영되지 않는다. source로 즉시 다시 읽게 만들어야 한다.

유래

스크립트 원본(source)을 새 프로세스로 실행하는 게 아니라, 지금 이 셸 안에서 직접 읽어 실행한다는 뜻에서 붙은 이름이다.

상황별 예제

01 설정 바로 적용

```
% source ~/.zshrc
```

alias나 PATH를 고친 뒤 새 창을 열 필요가 없다.

02 파이썬 가상환경 켜기

```
% source .venv/bin/activate
```

프롬프트 앞에 (.venv)가 붙으면 성공이다.

03 별명 파일 따로 관리

```
% source ~/.aliases
```

.zshrc 안에 이 줄을 넣어 두면 별명만 모은 파일을 따로 관리할 수 있다.

04 .env를 환경변수로

```
% set -a; source .env; set +a
```

set -a는 이후 만드는 변수를 자동으로 export한다.

history

셸 설정

history (기록)

한마디로 지금까지 입력한 명령을 번호와 함께 보여준다.

형식 **history**

```
~ % history | tail -3
512  cd notes
513  cp draft.md final.md
514  history | tail -3
```

유래

이름 그대로 기록. zsh는 입력한 명령을 ~/.zsh_history 파일에 저장해 두었다가 창을 새로 열어도 불러온다.

기록을 불러오는 기호와 키

옵션	원래 단어	뜻
!!	last command	바로 전에 실행한 명령을 그대로 다시 실행한다.
!번호	event number	history 목록에서 그 번호의 명령을 다시 실행한다.
!문자	prefix	그 글자로 시작하는 가장 최근 명령을 다시 실행한다. 예: !ssh
Control+R	r everse search	글자를 입력하면, 그 글자가 들어간 과거 명령을 찾아 보여준다.

01 전체 기록 보기

```
% history 1
```

zsh의 history는 최근 몇 개만 보여준다. 1을 붙이면 처음부터 전부.

02 번호로 다시 실행

```
% !513
```

513번 명령(cp draft.md final.md)이 그대로 다시 실행된다.

03 과거 명령 검색

```
% Control + R → ssh
```

ssh가 들어간 가장 최근 명령이 나타난다. 한 번 더 누르면 그 이전 것.

04 특정 명령만 골라 보기

```
% history 1 | grep git
```

git으로 했던 작업만 모아 본다.

꿀팁

Control+R 역검색은 한 번 익히면 다시는 돌아갈 수 없는 기능이다. 긴 명령을 다시 칠 일이 거의 사라진다.

man

셸 설정

manual

한마디로 모든 명령어의 공식 설명서를 연다.

형식 **man** 명령어

```
~ % man ls
LS(1)      General Commands Manual      LS(1)
NAME
    ls - list directory contents
→ q로 종료
```

유래

manual, 설명서. 1971년 켄 톰슨과 데니스 리치가 쓴 첫 유닉스 설명서(Unix Programmer's Manual)의 형식이 지금까지 그대로 이어진다. 설명서는 번호가 붙은 절로 나뉘며, 1절이 우리가 쓰는 명령어다.

주요 옵션

옵션	원래 단어	뜻
-k	k eyword	설명에 그 단어가 들어간 명령어들을 찾아준다. 이름을 모를 때 쓴다.
/단어	search	설명서 화면에서 그 단어가 있는 곳으로 이동한다.
q	q uit	설명서를 닫고 터미널로 돌아온다.

01 옵션 뜻 찾아보기

```
% man tar
```

화면에서 `/-z` 처럼 검색하면 그 옵션 설명으로 바로 간다.

02 이름을 모르는 명령 찾기

```
% man -k compress
```

압축과 관련된 명령이 설명과 함께 나열된다.

03 설명서가 길 때

```
% man ls | head -30
```

앞부분의 NAME과 SYNOPSIS만 봐도 쓰는 법의 절반은 알 수 있다.

꿀팁

macOS의 명령은 리눅스와 달리 `--help`를 모르는 경우가 많다. `ls --help`가 오류를 내면 망설이지 말고 `man ls`.

비하인드 스토리

문서에서 `ls(1)`, `printf(3)`처럼 이름 옆에 붙은 숫자는 man 설명서의 절 번호다. 50년 된 설명서의 흔적이 지금도 개발 문서 곳곳에 남아 있다.

11 빠른 참조 · 셸 환경과 도움말

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>alias ll='ls -lah'</code>	별명 붙이기
<code>export PATH="\$HOME/bin:\$PATH"</code>	PATH에 폴더 추가
<code>source ~/.zshrc</code>	설정 바로 적용
<code>history 1 grep 단어</code>	지난 명령 찾기
<code>Control + R</code>	지난 명령 역검색
<code>man 명령</code>	공식 설명서
<code>man -k 단어</code>	관련 명령 찾기

맥에서만 되는 것들

리눅스에는 없는 macOS만의 명령어. Finder, 클립보드, Spotlight, 잠자기, 음성까지 터미널에서 다룬다.

open

맥 전용

open (macOS 전용)

한마디로 **Finder에서 더블클릭한 것처럼 연다.**

형식 **open** 파일|폴더|주소

```
~ % open .
→ 지금 폴더가 Finder 창으로 열림
~ % open report.pdf
→ 미리보기 앱으로 열림
```

유래

이름 그대로 "열기". 맥의 뿌리인 NeXTSTEP 시절부터 이어진 macOS 전용 명령어이다. 리눅스에는 없어서 맥 사용자의 특권 같은 명령어이다.

주요 옵션

옵션	원래 단어	뜻
-a	application	기본 앱 대신 원하는 앱으로 연다. 예: -a Safari
-e	TextEdit	텍스트 편집기(TextEdit)로 연다.
-R	Reveal	파일을 열지 않고, Finder에서 그 파일이 있는 위치를 보여준다.
-n	new instance	이미 열려 있는 앱이라도 새로 하나 더 띄운다.

상황별 예제

01 터미널에서 Finder로

```
% open .
```

.은 현재 폴더. 터미널과 Finder를 오가는 가장 빠른 길이다.

02 VS Code로 프로젝트 열기

```
% open -a "Visual Studio Code" .
```

앱 이름에 공백이 있으면 따옴표로 감싼다.

03 웹사이트 열기

```
% open https://www.apple.com
```

기본 브라우저로 주소가 열린다.

04 파일 위치 보여주기

```
% open -R ~/Downloads/report.pdf
```

파일을 열지 않고 Finder에서 선택된 상태로 보여준다.

꿀팁

`open .`은 맥 사용자들이 가장 사랑하는 명령 중 하나다. 터미널 작업 중에 "이 폴더 Finder로 보고 싶다" 싶으면 바로 치면 된다.

pbcopy

맥 전용

pasteboard copy · pbpaste (macOS 전용)

한마디로 터미널과 클립보드(Cmd+C·Cmd+V)를 잇는다.

형식 명령 | pbcopy / pbpaste

```
~ % cat ~/.ssh/id_ed25519.pub | pbcopy
→ 공개키가 클립보드에 복사됨, Cmd+V로 붙여넣기
~ % pbpaste > note.txt
→ 클립보드 내용을 파일로 저장
```

유래

pasteboard copy. macOS는 클립보드를 붙여넣기 판(pasteboard)이라고 부른다. NeXTSTEP 시절부터 쓰던 용어가 명령어 이름에 남았다. 짝인 pbpaste는 반대로 클립보드 내용을 꺼내 온다.

짝을 이루는 두 명령

옵션	원래 단어	뜻
pbcopy	p aste b oard copy	앞 명령의 결과를 클립보드에 넣는다. 이후 Cmd+V로 어디든 붙여넣을 수 있다.
pbpaste	p aste b oard paste	Cmd+C로 복사해 둔 내용을 터미널로 꺼내 온다.

상황별 예제

01 GitHub에 SSH 키 등록

```
% pbcopy < ~/.ssh/id_ed25519.pub
```

마우스로 굵다가 한 글자 빠뜨리는 실수가 없다.

02 현재 경로 복사

```
% pwd | pbcopy
```

동료에게 폴더 위치를 알려줄 때.

03 복사한 목록 정리해 다시 복사

```
% pbpaste | sort -u | pbcopy
```

웹에서 복사한 목록의 중복을 없애고 정렬해서 다시 클립보드에 넣는다.

04 명령 결과 공유

```
% ls -la | pbcopy
```

결과를 그대로 메신저나 이슈에 붙여넣는다.

꿀팁

pbpaste | 명령 | pbcopy 조합을 기억해 두자. 클립보드에 있는 글을 터미널 명령으로 가공해 다시 클립보드로 돌려보내는, 맥에서만 가능한 요령이다.

mdfind

맥 전용

metadata find (macOS 전용)

한마디로 **Spotlight의 색인으로 순식간에 찾는다.**

형식 **mdfind "검색어"**

```
~ % mdfind "분기 보고서"
/Users/me/Documents/2026-Q3 분기 보고서.pdf
/Users/me/Desktop/보고서 초안.docx
```

유래

metadata find. md는 메타데이터(파일에 대한 정보)를 뜻한다. Cmd+Space로 쓰는 Spotlight가 미리 만들어 둔 색인을 터미널에서 검색하는 명령이다.

주요 옵션

옵션	원래 단어	뜻
-name	<u>name</u>	파일 내용은 보지 않고 이름에서만 찾는다.
-onlyin	<u>only in</u>	디스크 전체 대신 지정한 폴더 안에서만 찾는다.
-count	<u>count</u>	찾은 파일 목록 대신 개수만 알려준다.
-live	<u>live</u>	끝내지 않고 기다리다가, 조건에 맞는 파일이 새로 생기면 알려 준다.

상황별 예제

01 파일 이름으로

```
% mdfind -name "invoice"
```

이름에 invoice가 들어간 파일을 디스크 전체에서 찾는다.

02 특정 폴더에서 내용으로

```
% mdfind -onlyin ~/Documents "계약서"
```

문서 안에 "계약서"라는 단어가 있는 파일까지 찾는다.

03 PDF가 몇 개인가

```
% mdfind -count "kind:pdf"
```

kind: 뒤에 종류를 적으면 파일 형식으로 거른다.

꿀팁

find는 디스크를 직접 한 칸씩 뒤져 느리지만, mdfind는 색인을 보기 때문에 거의 즉시 끝난다. 대신 Spotlight에서 제외한 폴더는 찾지 못한다.

caffeinate

맥 전용

caffeinate (macOS 전용)

한마디로 맥이 잠들지 않게 커피를 먹인다.

형식 **caffeinate** [옵션] [명령]

```
~ % caffeinate -t 3600
→ 1시간 동안 잠자기 모드로 들어가지 않음
```

유래

caffeine(카페인)에 동사 어미 -ate를 붙인 애플의 말장난이다. "맥에게 커피를 먹인다"는 뜻으로, 2012년 OS X Mountain Lion에 들어왔다.

주요 옵션

옵션	원래 단어	뜻
-d	d isplay	화면이 꺼지지 않게 한다. 발표나 영상을 틀어 둘 때.
-i	i dle	화면은 꺼져도 맥 자체는 잠들지 않게 한다. 다운로드나 백업을 걸어 둘 때.
-t	t imeout	몇 초 동안만 깨어 있을지 정한다. 예: -t 3600은 1시간.
-w	w ait for PID	지정한 번호(PID)의 프로그램이 끝날 때까지 깨어 있다.

상황별 예제

01 긴 다운로드 동안만

```
% caffeinate -i curl -O https://example.com/big.iso
```

다운로드가 끝나면 자동으로 원래대로 돌아간다.

02 발표 중 화면 유지

```
% caffeinate -d
```

Control+C를 누를 때까지 화면이 꺼지지 않는다.

03 백업 스크립트 끝날 때까지

```
% caffeinate -i ./backup.sh
```

명령 앞에 붙이기만 하면 된다.

04 이미 실행 중인 작업을 지키기

```
% caffeinate -w 4821
```

PID 4821 프로그램이 끝날 때까지 깨어 있다.

비하인드 스토리

애플 엔지니어들의 유머 감각이 드러나는 몇 안 되는 명령어 이름이다. 두 글자로 말하는 사람들 사이에서 거의 유일하게 긴 이름을 당당히 쓰는 명령이기도 하다.

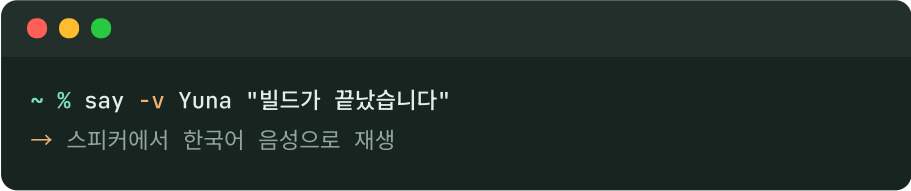
say

맥 전용

say (말하다, macOS 전용)

한마디로 맥이 글자를 소리 내어 읽는다.

형식 **say "문장"**



```
~ % say -v Yuna "빌드가 끝났습니다"
→ 스피커에서 한국어 음성으로 재생
```

유래

이름 그대로 "말하다". 1984년 첫 매킨토시 발표 무대에서 맥이 스스로 "Hello, I'm Macintosh"라고 인사했던 음성 합성 기술의 후손이다.

주요 옵션

옵션	원래 단어	뜻
-v	<u>v</u> oice	목소리를 고른다. 한국어 목소리는 Yuna.
-r	<u>r</u> ate	말하는 속도를 정한다. 숫자가 클수록 빠르다(1분에 읽는 단어 수).
-o	<u>o</u> utput file	소리 내어 읽는 대신 음성 파일로 저장한다.
-f	<u>f</u> ile	텍스트 파일의 내용을 통째로 읽어 준다.

01 작업이 끝나면 알려줘

```
% ./backup.sh; say "백업이 끝났어요"
```

;는 앞 명령이 끝나면 성공과 상관없이 다음 명령을 실행한다. 다른 일을 하다가 소리로 알 수 있다.

02 쓸 수 있는 목소리 목록

```
% say -v '?'
```

언어별 목소리와 예문이 나열된다.

03 음성 파일 만들기

```
% say -v Yuna -o hello.aiff "안녕하세요"
```

안내 방송용 음성 파일을 똑딱 만든다.

04 원고 소리 내어 검토

```
% say -v Yuna -f essay.txt
```

눈으로는 놓친 어색한 문장이 귀에는 걸린다.

꿀팁

긴 빌드나 다운로드 명령 끝에 ; say 완료 를 붙여 두는 습관을 들이면 터미널을 계속 지켜볼 필요가 없다.

12 빠른 참조 · 맥에서만 되는 것들

이 장에서 가장 자주 쓰는 형태만 모았다. 막힐 때는 이 쪽만 펼쳐 봐도 된다.

<code>open .</code>	지금 폴더를 Finder로
<code>open -a "앱 이름" 파일</code>	지정한 앱으로 열기
명령 <code>pbcopy</code>	결과를 클립보드로
<code>pbpaste > 파일</code>	클립보드를 파일로
<code>mdfind -name "단어"</code>	Spotlight로 빠르게 찾기
<code>caffeinate -i</code> 명령	작업 중 잠들지 않게
명령; <code>say "완료"</code>	끝나면 소리로 알림

두 글자로 문제 풀기

1부에서 익힌 단어들을 실제 상황에 이어 쓰는 법을 다룬다. 한 사람의 하루를 따라 프로젝트를 처음부터 끝까지 진행해 보고, 맥을 쓰다 흔히 만나는 문제 여덟 가지를 증상에서 원인, 해결, 확인까지 한 단계씩 풀어 본다. 모든 명령은 조각마다 뜻을 풀어 설명한다.

— 13장 · 따라 하기: 프로젝트 하나를 처음부터 끝까지

— 14장 · 문제 해결 사례집: 증상에서 해결까지

단어를 아는 것과 말할 줄 아는 것은 다르다.

이제 문장을 만들 차례다.

따라 하기: 프로젝트를 하나를 처음부터 끝까지

지금까지 단어를 하나씩 배웠다면, 이 장에서는 그 단어들이 문장처럼 이어지는 하루를 따라간다. 두 글자로 말하는 사람들이 실제로 어떻게 일하는지 보는 장이다.

지

지민 · 강사

강의 교안을 웹사이트로 만들어 수강생들과 공유하려고 한다. 터미널은 이 책으로 처음 배웠다.

오늘의 목표 빈 폴더에서 시작해 저녁까지 첫 버전을 서버에 올리고 백업까지 마치고.

이 장을 읽는 법

- **지금 상황에서 왜 이 일을 하는지** 먼저 읽는다.
- **터미널에서 실제로 입력한 명령과 돌아온 결과를** 본다.
- **명령어 해부**에서 한 줄을 조각(명령 · 옵션 · 경로 · 기호)으로 나눠 각각의 뜻을 확인한다.
- **흔한 실수로 실제로 막히는 지점을** 미리 알아 둔다.
- 맨 아래 쪽수를 따라가면 그 명령어의 사전 항목으로 돌아가 복습할 수 있다.
- 점선 상자의 git, npm은 이 책에서 다루지 않는 도구다. 흐름을 이해할 정도로만 설명한다.

지민의 하루 한눈에 보기

아홉 단계, 스물다섯 개의 명령어. 점선 칩은 이 책 밖의 도구다.

- 09:00 • STEP 1 **작업할 자리 만들기** p.144
 - mkdir cd pwd
- 09:05 • STEP 2 **도구가 준비됐는지 확인하기** p.146
 - which node
- 09:10 • STEP 3 **빠대 세우기** p.148
 - cp mkdir touch ls
- 09:20 • STEP 4 **설정 파일 채우기** p.151
 - echo cat
- 09:30 • STEP 5 **개발 서버 켜기, 그리고 첫 번째 막힘** p.153
 - lsof kill npm
- 11:00 • STEP 6 **작업하면서 수시로** p.155
 - grep find wc sed
- 15:00 • STEP 7 **올리기 전 점검** p.158
 - grep find du sort
- 16:00 • STEP 8 **저장하고 내보내기** p.161
 - ssh df rsync tar git npm
- 18:00 • STEP 9 **내일을 위한 정리** p.165
 - echo alias source open say

STEP 1

09:00

작업할 자리 만들기

지금 상황 지민은 앞으로 만들 프로젝트를 한곳에 모아 두기로 했다. 홈 폴더 아래 Projects 폴더를 만들고, 그 안에 이번 강의 사이트용 폴더를 만든 뒤 들어간다.

```
~ % mkdir -p ~/Projects/lecture-site
(출력 없음)
~ % cd ~/Projects/lecture-site
→ 프롬프트가 lecture-site % 로 바뀜
lecture-site % pwd
/Users/jimin/Projects/lecture-site
```

명령어 해부 · 한 줄씩, 조각마다

```
% mkdir -p ~/Projects/lecture-site
```

mkdir

폴더를 만드는 명령 (make directory)

-p

중간 폴더인 Projects가 아직 없어도 함께 만들라는 옵션

~

내 홈 폴더 /Users/jimin을 줄여 쓴 기호

/Projects/lecture-site

홈 아래 Projects, 그 아래 lecture-site라는 경로

결과 읽기 아무 말도 없으면 성공이다. 유닉스 명령은 잘 되면 조용하고, 문제가 있을 때만 말한다.

```
% cd ~/Projects/lecture-site
```

cd 다른 폴더로 이동하는 명령 (change directory)

~/Projects/lecture 방금 만든 폴더의 경로
-site

결과 읽기 출력은 없지만 프롬프트의 위치 표시가 lecture-site로 바뀐다. 이제부터 모든 명령은 이 폴더 안에서 실행된다.

```
% pwd
```

pwd 지금 있는 폴더의 전체 경로를 출력 (print working directory)

결과 읽기 프롬프트는 공간을 아끼려고 마지막 폴더 이름만 보여준다. 정확한 위치가 궁금할 때는 pwd로 확인한다.

흔한 실수

mkdir만 하고 cd를 잊으면 이후에 만드는 파일이 모두 홈 폴더에 들어간다. 새 폴더를 만들면 바로 들어가는 습관을 들이거나, mkdir -p 폴더 && cd 폴더 처럼 한 줄로 이어 쓴다.

이 단계에서 쓴 명령어 · **mkdir** p.23 · **cd** p.16 · **pwd** p.14

STEP 2

09:05

도구가 준비됐는지 확인하기

지금 상황 웹사이트를 만들려면 Node.js와 git이 필요하다. 예전에 설치한 기억은 있지만 정말 있는지, 어디에 있는지 확인한다.

```
lecture-site % which node git
/opt/homebrew/bin/node
/usr/bin/git
lecture-site % node -v
v22.9.0
```

명령어 해부 · 한 줄씩, 조각마다

% which node git

which

명령어가 실제로 어디 있는 프로그램인지 찾는 명령

node git

찾을 이름 두 개. 공백으로 나열하면 한 번에 여러 개를 물을 수 있다

결과 읽기 두 줄이 나왔으니 둘 다 설치돼 있다. /opt/homebrew/bin은 Homebrew로 설치한 프로그램이 모이는 곳이고, /usr/bin은 macOS에 원래 들어 있는 프로그램의 자리다. 아무 줄도 나오지 않는 이름이 있다면 설치되지 않았거나 PATH에 빠져 있는 것이다.

책 밖의 도구 · 다음 단계에서 배울 명령

node -v — Node.js의 버전을 확인한다. -v는 version의 약자로, 대부분의 프로그램이 -v나 --version으로 버전을 알려준다. 튜토리얼이 요구하는 버전과 맞는지 비교할 때 쓴다.

흔한 실수

분명히 설치했는데 `command not found`가 뜬다면 설치 위치가 PATH에 없는 경우가 대부분이다. `which -a`로 같은 프로그램이 여러 개 있는지 보고, `export` 항목의 PATH 추가 방법을 따라 한다.

이 단계에서 쓴 명령어 · **which** p.58

지금 상황 동료가 쓰던 시작 템플릿에서 설정 파일(package.json)을 복사해 오고, 사이트에 필요한 폴더와 빈 파일을 만든다. 마지막으로 제대로 만들어졌는지 눈으로 확인한다.

```
lecture-site % cp ~/Templates/vite-starter/package.json .
```

(출력 없음)

```
lecture-site % mkdir -p src/{components,assets} docs
```

(출력 없음)

```
lecture-site % touch README.md .gitignore
```

(출력 없음)

```
lecture-site % ls -la
```

```
drwxr-xr-x  7 jimin  staff  224  9 30 09:12 .
-rw-r--r--  1 jimin  staff    0  9 30 09:12 .gitignore
-rw-r--r--  1 jimin  staff    0  9 30 09:12 README.md
drwxr-xr-x  2 jimin  staff   64  9 30 09:12 docs
-rw-r--r--  1 jimin  staff  612  9 30 09:11 package.json
drwxr-xr-x  4 jimin  staff  128  9 30 09:12 src
```

```
% cp ~/Templates/vite-starter/package.json .
```

cp	파일을 복사하는 명령 (copy)
~/Templates/vite-starter/package.json	복사할 원본 파일의 경로
.	지금 있는 폴더. 대상이 폴더이면 원래 이름 그대로 그 안에 복사된다

결과 읽기 조용하면 성공이다. package.json은 이 프로젝트에 어떤 부품이 필요한지 적어 둔 목록이다. 5단계에서 이 목록을 보고 부품을 설치한다.

```
% mkdir -p src/{components,assets} docs
```

-p	상위 폴더 src가 없어도 함께 만든다
src/{components,assets}	중괄호 확장. 셸이 src/components와 src/assets 두 경로로 펼쳐서 mkdir에 넘겨준다
docs	세 번째로 만들 폴더. 공백으로 구분해 여러 개를 한 번에 적는다

결과 읽기 한 줄로 폴더 네 개(src, src/components, src/assets, docs)가 생긴다.

```
% touch README.md .gitignore
```

touch	파일이 없으면 빈 파일을 만드는 명령
README.md	프로젝트 설명서. GitHub에 올리면 첫 화면에 보인다
.gitignore	git에 올리지 않을 파일 목록. 점(.)으로 시작해서 숨김 파일이 된다

결과 읽기 크기 0바이트짜리 빈 파일 두 개가 생긴다. 내용은 다음 단계에서 채운다.

```
% ls -la
```

ls	폴더 안의 내용을 나열하는 명령 (list)
-l	long. 권한·주인·크기·날짜까지 한 줄에 하나씩 자세히
-a	all. 이름이 점으로 시작하는 숨김 파일까지 모두

결과 읽기 맨 앞 글자가 d면 폴더, -면 파일이다. 크기 칸의 0은 방금 만든 빈 파일, 612는 복사해 온 package.json의 크기(바이트)다. .gitignore가 보이는 건 -a 덕분이다. -a를 빼면 목록에서 사라진다.

흔한 실수

{components, assets}처럼 심표 뒤에 공백을 넣으면 중괄호 확장이 깨져서 '{components,'와 'assets}'라는 엉뚱한 폴더가 생긴다. 중괄호 안에는 공백을 넣지 않는다.

이 단계에서 쓴 명령어 · **cp** p.33 · **mkdir** p.23 · **touch** p.28 · **ls** p.19

지금 상황 git에 올리면 안 되는 것들을 .gitignore에 적는다. 수백 MB짜리 부품 폴더(node_modules)와 비밀번호를 담은 파일(.env)이다. 편집기를 열지 않고 터미널에서 바로 쓴다.

```
lecture-site % echo "node_modules" >> .gitignore
(출력 없음)
lecture-site % echo ".env" >> .gitignore
(출력 없음)
lecture-site % cat .gitignore
node_modules
.env
```

명령어 해부 · 한 줄씩, 조각마다

```
% echo "node_modules" >> .gitignore
```

echo	따옴표 안의 글자를 그대로 내보내는 명령
"node_modules"	내보낼 글자. 설치한 부품이 모이는 폴더의 이름
>>	결과를 화면 대신 파일 맨 끝에 덧붙이라는 기호
.gitignore	글자를 덧붙일 파일

결과 읽기 화면에 아무것도 나오지 않는다. 화면에 나올 글자가 파일 속으로 들어갔기 때문이다.


```
% echo ".env" >> .gitignore
```

".env"

API 키나 비밀번호를 적어 두는 파일의 이름. 절대 인터넷에 올리면 안 된다

결과 읽기 첫 줄 아래에 두 번째 줄이 덧붙는다.

```
% cat .gitignore
```

cat

파일 내용을 화면에 그대로 보여주는 명령
(concatenate)

.gitignore

내용을 볼 파일

결과 읽기 두 줄이 적은 순서대로 들어 있다. 편집기를 열지 않고도 파일 내용을 확인할 수 있다.

흔한 실수

>> 대신 >를 쓰면 파일을 매번 새로 덮어써서 마지막 한 줄만 남는다. 두 번째 echo에서 >를 쓰면 node_modules 줄이 사라져, 나중에 수백 MB가 통째로 git에 올라가는 사고가 난다.

이 단계에서 쓴 명령어 · **echo** p.49 · **cat** p.41

STEP 5**09:30 개발 서버 켜기, 그리고 첫 번째 막힘**

지금 상황 부품을 설치하고 개발 서버를 켜려는데 오류가 난다. 어제 켜 두고 잊어버린 서버가 아직 같은 포트(문 번호)를 붙잡고 있었다. 범인을 찾아 끄고 다시 켜다.

```
lecture-site % npm install
added 212 packages in 9s
lecture-site % npm run dev
Error: Port 5173 is already in use
lecture-site % lsof -i :5173
COMMAND  PID   USER   FD   TYPE    NAME
node     4821  jimin  23u  IPv6  *:5173 (LISTEN)
lecture-site % kill 4821
(출력 없음)
lecture-site % npm run dev
Local:   http://localhost:5173/
```

명령어 해부 · 한 줄씩, 조각마다

% lsof -i :5173

lsof	어떤 프로그램이 무엇을 열고 있는지 보여주는 명령 (list open files)
-i	그중 네트워크 연결만 보라는 옵션 (internet)
:5173	5173번 포트만. 콜론 뒤에 포트 번호를 적는다

결과 읽기 COMMAND 칸의 node가 프로그램 이름, PID 칸의 4821이 그 프로그램의 고유 번호다. 맨 끝의 LISTEN은 "손님을 기다리는 중"이라는 뜻이다. 어제 끄지 않은 개발 서버가 아직 문을 열고 있었다.

% kill 4821

kill

프로그램에 신호를 보내는 명령. 숫자 없이 쓰면 "정리하고 끝내 주세요"라는 정상 종료 요청

4821

신호를 받을 프로그램의 PID. 바로 앞 `ls`에서 알아낸 번호

결과 읽기 조용하면 성공이다. 그래도 꺼지지 않으면 강제 종료인 `kill -9 4821`을 쓴다.

책 밖의 도구 · 다음 단계에서 배울 명령

npm install — `package.json`에 적힌 부품(패키지)을 인터넷에서 받아 `node_modules` 폴더에 설치한다. "212개를 9초 만에 설치했다"는 결과다.

npm run dev — 개발용 서버를 켜다. 코드를 고치면 브라우저에 바로 반영된다. 마지막 줄의 주소를 브라우저에 넣으면 사이트가 보인다.

흔한 실수

포트 오류를 보고 맥을 재부팅하는 사람이 많다. 두 줄이면 끝나는 일이다. 익숙해지면 `kill $(ls -t -i :5173)` 한 줄로 줄일 수 있다.

이 단계에서 쓴 명령어 · **ls** p.107 · **kill** p.104

STEP 6

11:00

작업하면서 수시로

지금 상황 오전 내내 슬라이드 페이지를 만들었다. 중간중간 남은 할 일을 찾고, 만든 파일 수를 세고, 작년 자료에서 가져온 문서의 연도를 한꺼번에 고친다.

```
lecture-site % grep -rn "TODO" src/
src/App.tsx:42:// TODO: 3장 슬라이드 추가
src/Slide.tsx:17:// TODO: 이미지 크기 조정
lecture-site % find src -name "*.tsx" | wc -l
14
lecture-site % sed -i '' 's/2025/2026/g' docs/*.md
(출력 없음)
lecture-site % grep -rn "2025" docs/
(출력 없음)
```

명령어 해부 · 한 줄씩, 조각마다

```
% grep -rn "TODO" src/
```

grep	파일 속에서 글자를 찾는 명령
-r	지정한 폴더와 그 아래 모든 파일에서 찾는다 (recursive)
-n	찾은 줄이 몇 번째 줄인지 번호를 붙인다 (line number)
"TODO"	찾을 글자. 개발자들이 "나중에 할 일"을 표시할 때 쓰는 관례적인 단어
src/	찾을 범위

결과 읽기 결과는 "파일 이름:줄 번호:그 줄의 내용" 형식이다. 남은 할 일이 두 개이고, 편집기에서 App.tsx의 42번째 줄로 바로 가면 된다.

```
% find src -name "*.tsx" | wc -l
```

find src	src 폴더 안을 뒤진다
-name "*.tsx"	이름이 .tsx로 끝나는 파일만. *는 '아무 글자나'
 	파이프. 앞 명령의 결과를 뒤 명령에 넘긴다
wc -l	넘겨받은 결과의 줄 수를 센다. 파일이 한 줄에 하나씩이므로 곧 파일 개수

결과 읽기 화면 부품(.tsx) 파일이 14개라는 뜻이다. 두 명령을 파이프로 이으면 "찾아서 세기"라는 새 기능이 생긴다.

```
% sed -i '' 's/2025/2026/g' docs/*.md
```

sed	흘러가는 글을 편집하는 명령 (stream editor)
-i ''	결과를 화면에 보이는 대신 원본 파일을 직접 고친다. macOS에서는 뒤에 빈 따옴표가 꼭 필요하다
's/2025/2026/g'	바꾸기 규칙. s는 substitute(바꾸기), 2025를 2026으로, g는 한 줄에 여러 개 있어도 모두
docs/*.md	docs 폴더의 모든 .md 파일

결과 읽기 조용하면 성공이다. 문서가 몇 개든 한 번에 고쳐진다.

```
% grep -rn "2025" docs/
```

"2025"	남아 있는 옛 연도가 있는지 다시 찾는다
---------------	------------------------

결과 읽기 아무것도 나오지 않으면 모두 바뀌었다는 뜻이다. 고친 뒤 grep으로 확인하는 것까지가 한 세트다.

흔한 실수

`sed -i`를 처음부터 쓰면 되돌릴 수 없다. 먼저 `-i` 없이 실행해 화면으로 결과를 확인하거나, `-i .bak`을 써서 원본을 `.bak` 파일로 남겨 둔다.

이 단계에서 쓴 명령어 · **grep** p.55 · **find** p.52 · **wc** p.61 · **sed** p.65

지금 상황 오후에 첫 버전이 완성됐다. 인터넷에 올리기 전에 세 가지를 점검한다. 비밀키가 코드에 섞이지 않았는지, 맥이 몰래 만든 찌꺼기 파일은 없는지, 어떤 폴더가 무거운지.

```
lecture-site % grep -rI "API_KEY" . --exclude-dir=node_modules
./env
lecture-site % find . -name ".DS_Store" -delete
(출력 없음)
lecture-site % du -sh * | sort -rh
187M    node_modules
1.2M    src
24K     docs
4.0K    package.json
0B      README.md
```

```
% grep -rل "API_KEY" . --exclude-dir=node_modules
```

-r	모든 하위 폴더까지
-ل	내용은 빼고 그 글자가 든 파일 이름만 (list files)
"API_KEY"	비밀키 변수에 흔히 붙이는 이름
.	현재 폴더 전체
--exclude-dir=node_modules	설치한 부품 폴더는 건너뛰다. 파일이 수만 개라 느리고, 내가 쓴 코드도 아니다

결과 읽기 .env 한 곳에서만 나왔다. .env는 4단계에서 .gitignore에 넣었으니 올라가지 않는다. 만약 src 안의 파일 이름이 나왔다면 코드에 비밀키가 직접 적힌 것이니 반드시 고쳐야 한다.

```
% find . -name ".DS_Store" -delete
```

find .	현재 폴더 전체를 뒤진다
-name ".DS_Store"	Finder가 폴더마다 몰래 만드는 설정 파일의 이름
-delete	찾자마자 지운다

결과 읽기 조용하면 끝이다. 이 파일은 다른 사람에게 쓸모가 없고 폴더 구조를 노출하기도 해서 올리기 전에 지우는 것이 좋다.


```
% du -sh * | sort -rh
```

du	용량을 재는 명령 (disk usage)
-s	하위 폴더를 나열하지 않고 항목마다 합계 한 줄씩
-h	K, M, G 같은 읽기 쉬운 단위로
*	현재 폴더의 모든 항목
sort -rh	단위를 이해하면서(-h) 큰 것부터(-r) 정렬

결과 읽기 node_modules가 187M로 압도적으로 크고, 지민이 직접 쓴 코드(src)는 1.2M뿐이다. 그래서 node_modules를 .gitignore에 넣어 올리지 않는 것이다.

흔한 실수

grep 결과가 비어 있다고 안심하기 전에 검색어를 한 번 더 생각하자. 서비스마다 SECRET, TOKEN, PASSWORD처럼 다른 이름을 쓴다. grep -rIE "API_KEY|SECRET|TOKEN"처럼 여러 단어를 한 번에 찾으면 더 안전하다.

이 단계에서 쓴 명령어 · **grep** p.55 · **find** p.52 · **du** p.113 · **sort** p.63

지금 상황 작업 기록을 남기고, 배포용 파일을 만들어 서버에 올린다. 올리기 전에 서버에 자리가 충분한지 확인하고, 마지막으로 날짜가 붙은 백업을 만든다.

```
lecture-site % git init && git add . && git commit -m "First
lecture site"
[main (root-commit) a1b2c3d] First lecture site
 9 files changed, 214 insertions(+)
lecture-site % npm run build
built in 1.84s
lecture-site % ssh jimin@myserver "df -h /var/www"
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       40G   12G   28G   30% /
lecture-site % rsync -avz dist/
jimin@myserver:/var/www/lecture/
sending incremental file list
index.html
assets/index-4f2a.js
sent 1,204,881 bytes  received 412 bytes
lecture-site % tar -czf ~/Backups/lecture-$(date
+%Y%m%d).tar.gz --exclude=node_modules .
(출력 없음)
```

```
% ssh jimin@myserver "df -h /var/www"
```

ssh	다른 컴퓨터에 암호화된 연결로 접속하는 명령 (secure shell)
jimin@myserver	서버의 사용자 이름@서버 주소
"df -h /var/www"	접속해서 실행할 명령. 따옴표 안의 명령만 실행하고 바로 내 맥으로 돌아온다
df -h	디스크의 남은 공간을 읽기 쉬운 단위로 보여준다 (disk free)

결과 읽기 서버 디스크 40G 중 28G가 남아 있다(Avail). 1.2MB짜리 사이트를 올리기에 충분하다. 서버에 들어가 살펴보고 나오는 과정을 한 줄로 줄였다.

```
% rsync -avz dist/ jimin@myserver:/var/www/lecture/
```

rsync	바뀐 부분만 골라 복사하는 동기화 명령 (remote sync)
-a	권한·날짜까지 원본 그대로 (archive)
-v	보내는 파일 목록을 보여준다 (verbose)
-z	압축해서 보내 전송 시간을 줄인다 (zip)
dist/	보낼 폴더. 끝의 /는 "폴더 안의 내용물"이라는 뜻
jimin@myserver:/var/www/lecture/	받는 곳. 콜론 뒤가 서버 안의 경로

결과 읽기 보낸 파일 목록과 전송량(약 1.2MB)이 나온다. 다음에 다시 배포하면 바뀐 파일만 전송되어 훨씬 빨리 끝난다.

```
% tar -czf ~/Backups/lecture-$(date +%Y%m%d).tar.gz --exclude=node_modules .
```

tar	여러 파일을 하나로 묶는 명령 (tape archive)
-c	새 압축 파일을 만든다 (create)
-z	gzip으로 압축한다
-f	바로 뒤에 압축 파일 이름이 온다 (file)
\$(date +%Y%m%d)	date 명령의 결과, 즉 오늘 날짜 20260930이 이 자리에 들어간다
--exclude=node_modules	다시 설치할 수 있는 부품 폴더는 뺀다
.	묶을 대상. 현재 폴더 전체

결과 읽기 ~/Backups/lecture-20260930.tar.gz 파일이 생긴다. 날짜가 이름에 들어가 있어서 백업이 쌓여도 헷갈리지 않는다.

책 밖의 도구 · 다음 단계에서 배울 명령

git init && git add . && git commit -m "..." — git은 작업 기록을 저장하는 도구다. init은 기록을 시작하고, add .은 모든 파일을 기록 대상에 올리고, commit은 메시지와 함께 지금 상태를 저장한다. &&는 앞 명령이 성공했을 때만 다음 명령을 실행한다.

npm run build — 사이트를 배포용으로 가볍게 만들어 dist 폴더에 담는다. 서버에 올리는 것은 이 dist 폴더다.

흔한 실수

rsync에서 dist/ 대신 dist라고 쓰면 서버에 폴더가 한 겹 더 생겨 /var/www/lecture/dist가 된다. 사이트 주소가 바뀌어 버린다. 끝의 슬래시 하나가 결과를 바꾼다. 불안하면 -n을 붙여 예행연습부터 한다.

이 단계에서 쓴 명령어 · **ssh** p.92 · **df** p.115 · **rsync** p.96 · **tar** p.79

지금 상황 하루를 마무리하며 내일 더 빨리 시작할 수 있게 준비한다. 긴 폴더 경로에 별명을 붙이고, Finder로 결과물을 한 번 훑어본 뒤, 맥에게 퇴근을 알리게 한다.

```
lecture-site % echo "alias lec='cd ~/Projects/lecture-site'" >>
~/.zshrc
(출력 없음)
lecture-site % source ~/.zshrc
(출력 없음)
~ % lec
→ 어디에 있든 lecture-site 폴더로 이동
lecture-site % open .
→ Finder 창에 lecture-site 폴더가 열림
lecture-site % say -v Yuna "오늘 작업 끝"
→ 스피커에서 "오늘 작업 끝"
```

명령어 해부 · 한 줄씩, 조각마다

```
% echo "alias lec='cd ~/Projects/lecture-site'" >> ~/.zshrc
```

alias lec='...'	lec라는 짧은 별명이 따옴표 안의 명령을 대신하게 한다
------------------------	---------------------------------

큰따옴표 속 작은따옴표	alias 정의에도 따옴표가 필요해서, 바깥은 큰따옴표로 안은 작은따옴표로 겹쳐 쓴다
---------------------	---

>> ~/.zshrc	터미널이 켜질 때마다 읽는 설정 파일의 맨 끝에 덧붙인다
--------------------------	---------------------------------

결과 읽기 조용하면 설정 파일에 한 줄이 추가된 것이다. 하지만 이미 열려 있는 이 창은 아직 모른다.

% **source ~/.zshrc**

source	파일을 지금 이 창에서 다시 읽어 즉시 적용하는 명령
---------------	-------------------------------

~/.zshrc	방금 한 줄을 추가한 설정 파일
-----------------	-------------------

결과 읽기 이제 이 창에서도 `lec`가 통한다. 새 창을 열 필요가 없다.

% **open .**

open	Finder에서 더블클릭한 것처럼 여는 macOS 전용 명령
-------------	-----------------------------------

.	지금 폴더
----------	-------

결과 읽기 터미널에서 작업한 결과를 Finder 창으로 눈으로 확인한다.

% **say -v Yuna "오늘 작업 끝"**

say	글자를 소리 내어 읽는 macOS 전용 명령
------------	--------------------------

-v Yuna	한국어 목소리 Yuna를 고른다 (voice)
----------------	---------------------------

"오늘 작업 끝"	읽을 문장
------------------	-------

결과 읽기 맥이 한국어로 하루의 끝을 알려준다. 긴 작업 끝에 붙여 두면 알림으로도 쓸 수 있다.

흔한 실수

`source`를 잊고 `lec`를 치면 `command not found`가 뜬다. 설정 파일을 고쳤다면 `source`를 실행하거나 새 창을 연다.

이 단계에서 쓴 명령어 · **echo** p.49 · **alias** p.118 · **source** p.122 · **open** p.129 · **say** p.138

오늘 쓴 명령어 25개

하루 동안 이 책의 50개 명령어 중 25개를 썼다. 쪽수를 따라가 사전 항목에서 다른 옵션도 함께 익혀 보자.

mkdir	STEP 1, 3	p.23
cd	STEP 1	p.16
pwd	STEP 1	p.14
which	STEP 2	p.58
cp	STEP 3	p.33
touch	STEP 3	p.28
ls	STEP 3	p.19
echo	STEP 4, 9	p.49
cat	STEP 4	p.41
lsof	STEP 5	p.107
kill	STEP 5	p.104
grep	STEP 6, 7	p.55
find	STEP 6, 7	p.52
wc	STEP 6	p.61
sed	STEP 6	p.65

du	STEP 7	p.113
sort	STEP 7	p.63
ssh	STEP 8	p.92
df	STEP 8	p.115
rsync	STEP 8	p.96
tar	STEP 8	p.79
alias	STEP 9	p.118
source	STEP 9	p.122
open	STEP 9	p.129
say	STEP 9	p.138

처음 터미널을 열었을 때 ls, cd, mv는 암호처럼 보였다. 오늘 지민은 그 두 글자들로 폴더를 만들고, 막힌 곳을 뚫고, 사이트를 세상에 내보냈다. 다음 장에서는 누구에게나 한 번쯤 찾아오는 문제들을 같은 방식으로 풀어 본다.

문제 해결 사례집: 증상에서 해결까지

맥을 쓰다 보면 누구에게나 한 번쯤 찾아오는 문제 여덟 가지를 골랐다. 모든 사례는 같은 네 박자로 풀어 간다. 추측하지 말고 명령으로 확인하는 습관이 이 장의 진짜 주제다.

- 1 **진단** 증상을 숫자와 이름으로 바꾼다. "느리다"가 아니라 "PID 5123이 CPU 298%"로.
- 2 **원인** 범위를 좁혀 가며 진짜 이유를 찾는다. 한 번에 하나씩, 결과를 보고 다음 명령을 고른다.
- 3 **해결** 되돌릴 수 있는 방법부터 쓴다. 지우기 전에 복사하고, 강제하기 전에 부탁한다.
- 4 **확인** 진단 때와 같은 명령으로 다시 켜다. 같은 자로 재야 나아졌는지 알 수 있다.

이 장의 사례

- CASE 1 디스크 공간이 부족하다는 경고가 뜬다 p.170
- CASE 2 맥이 갑자기 느려지고 팬이 요란하게 돈다 p.175
- CASE 3 외장하드가 "사용 중"이라며 추출되지 않는다 p.180
- CASE 4 스크립트를 실행하면 Permission denied가 뜬다 p.184
- CASE 5 분명 설치했는데 command not found가 뜬다 p.188
- CASE 6 어제 받은 파일이 어디 있는지 모르겠다 p.192
- CASE 7 인터넷이 안 된다. 어디가 문제일까 p.195
- CASE 8 문서 수십 개에 같은 오타가 있다 p.199

CASE 1

디스크 공간이 부족하다는 경고가 뜬다

증상

사진을 저장하려는데 "디스크가 거의 가득 찼습니다"라는 알림이 뜬다. 무엇이 공간을 차지하는지 Finder로는 도무지 알 수가 없다.

1 진단

2 원인

3 해결

4 확인

1 진단 정말로 얼마나 찼는지 숫자로 확인한다

먼저 느낌이 아니라 숫자를 본다. macOS는 시스템 파일과 내 데이터를 나눠 담는데, 내 파일이 들어 있는 곳은 /System/Volumes/Data다.

```
~ % df -h /System/Volumes/Data
Filesystem      Size  Used Avail Capacity  Mounted on
/dev/disk3s5    460Gi  438Gi   12Gi    98%
/System/Volumes/Data
```

명령어 해부

% df -h /System/Volumes/Data

df	디스크의 전체·사용·남은 공간을 보여주는 명령 (disk free)
-h	바이트 숫자 대신 Gi(기가바이트) 같은 읽기 쉬운 단위로
/System/Volumes/Data	내 문서·사진·앱 데이터가 실제로 저장되는 영역

결과 읽기 460Gi 중 438Gi를 쓰고 12Gi만 남았다(Avail). Capacity 98%, 경고가 뜰 만하다. 일부 칸은 생략했다.

2 원인 어느 폴더가 무거운지 좁혀 간다

홈 폴더를 한 단계씩 내려가며 큰 폴더를 찾는다. 가장 큰 곳으로 들어가 같은 방법을 반복하면 범인에 금방 닿는다.

```
% du -h -d 1 ~ 2>/dev/null | sort -rh | head -5
312G    /Users/jimin
148G    /Users/jimin/Movies
 96G    /Users/jimin/Library
 41G    /Users/jimin/Downloads
 18G    /Users/jimin/Documents

% find ~/Movies ~/Downloads -size +2G
/Users/jimin/Movies/강의녹화/0912_1강.mov
/Users/jimin/Movies/강의녹화/0919_2강.mov
/Users/jimin/Downloads/ubuntu-24.04.iso
```

명령어 해부

```
% du -h -d 1 ~ 2>/dev/null | sort -rh | head -5
```

du	폴더가 차지한 용량을 재는 명령 (disk usage)
-h	K·M·G 단위로 읽기 쉽게
-d 1	홈 바로 아래 폴더까지만 나눠서 합계를 낸다 (depth)
~	내 홈 폴더
2>/dev/null	"권한 없음" 같은 오류 메시지를 버리는 곳(/dev/null)으로 보낸다. 결과만 깔끔하게 보인다
 sort -rh	단위를 이해하며(-h) 큰 것부터(-r) 정렬
 head -5	그중 위에서 5줄만

결과 읽기 첫 줄은 홈 전체의 합계다. 그 아래에서 Movies(148G)가 가장 크다. 범인이 있는 방을 찾았다.

```
% find ~/Movies ~/Downloads -size +2G
```

find	조건에 맞는 파일을 찾는 명령
~/Movies ~/Downloa	뒤질 폴더 두 곳. 공백으로 나열한다
ds	
-size +2G	2GB보다 큰 파일만

결과 읽기 강의 녹화 파일들과 예전에 받아 둔 설치 이미지(.iso)가 나왔다. 녹화는 보관해야 하지만 맥 안에 둘 필요는 없고, .iso는 이미 쓸 일이 끝났다.

3 해결 옮길 것은 옮기고, 버릴 것은 버린다

녹화 파일은 외장하드로 옮긴다. 이때 바로 지우지 말고 복사 → 똑같은지 확인 → 원본 삭제 순서를 지킨다. 설치 이미지는 휴지통으로 보낸다.

```
~ % rsync -av ~/Movies/강의녹화/ /Volumes/Backup/강의녹화/
sending incremental file list
0912_1강.mov
0919_2강.mov
sent 96,412,904,221 bytes
~ % diff -rq ~/Movies/강의녹화 /Volumes/Backup/강의녹화
(출력 없음)
~ % rm -r ~/Movies/강의녹화
(출력 없음)
~ % mv ~/Downloads/ubuntu-24.04.iso ~/.Trash/
(출력 없음)
```

```
% rsync -av ~/Movies/강의녹화/ /Volumes/Backup/강의녹화/
```

rsync 복사 도중 끊겨도 이어서 할 수 있는 동기화 명령

-a 날짜·권한까지 원본 그대로

-v 복사하는 파일 이름을 보여준다

~/Movies/강의녹화/ 끝의 /는 "폴더 안의 내용물을"이라는 뜻

/Volumes/Backup/강의녹화/ 외장하드(이름 Backup) 안의 목적지. 연결한 디스크는 /Volumes 아래에 나타난다

결과 읽기 약 96GB가 복사됐다. 대용량 복사는 cp보다 rsync가 안전하다. 중간에 끊겨도 같은 명령을 다시 실행하면 남은 것만 복사한다.

```
% diff -rq ~/Movies/강의녹화 /Volumes/Backup/강의녹화
```

diff 두 대상을 비교하는 명령

-r 폴더 안의 파일까지 모두 비교

-q 어디가 다른지는 빼고 다른 파일이 있는지만 알려준다

결과 읽기 아무것도 나오지 않으면 두 폴더가 완전히 같다는 뜻이다. 이 확인을 거쳐야 원본을 지워도 된다.

```
% rm -r ~/Movies/강의녹화
```

rm 휴지통을 거치지 않고 지우는 명령

-r 폴더와 그 안의 파일까지 모두

결과 읽기 원본이 사라진다. 되돌릴 수 없으므로 반드시 바로 앞의 diff로 복사본을 확인한 다음에 실행한다.

```
% mv ~/Downloads/ubuntu-24.04.iso ~/.Trash/
```

mv

파일을 옮기는 명령

~/ .Trash/

Finder 휴지통의 실제 위치

결과 읽기 rm 대신 휴지통으로 보냈으니 필요하면 되살릴 수 있다. 단, 휴지통을 비워야 공간이 실제로 생긴다.

4 확인 공간이 돌아왔는지 다시 쟀다

Finder에서 휴지통을 비운 뒤, 처음과 같은 명령으로 다시 확인한다. 해결은 같은 측정으로 끝내야 한다.

```
~ % df -h /System/Volumes/Data
Filesystem      Size  Used Avail Capacity  Mounted on
/dev/disk3s5    460Gi 290Gi 160Gi    65%
/System/Volumes/Data
```

다음부터는

다운로드 폴더는 공간 도둑의 단골 은신처다. 한 달에 한 번 `find ~/Downloads -mtime +30` 으로 30일 넘게 방치된 파일을 확인하고 정리하자.

이 사례에서 쓴 명령어 · **df** p.115 · **du** p.113 · **sort** p.63 · **head** p.45 · **find** p.52 · **rsync** p.96 · **diff** p.110 · **rm** p.30 · **mv** p.36

CASE 2 맥이 갑자기 느려지고 팬이 요란하게 돈다

증상

아무것도 하지 않는데 팬이 쉬지 않고 돌고, 창 전환도 버벅인다. 무엇이 맥을 괴롭히고 있을까.

1 진단

2 원인

3 해결

4 확인

1 진단 CPU를 가장 많이 쓰는 프로그램을 찾는다

느려졌다면 거의 항상 누군가 CPU를 독차지하고 있다. 사용량 순으로 줄 세워 맨 위를 본다.

```
~ % ps aux -r | head -4
```

USER	PID	%CPU	%MEM	...	COMMAND
jimin	5123	298.4	3.1	...	node /Users/jimin/old-project/server.js
jimin	612	11.8	1.2	...	WindowServer
jimin	4410	6.2	4.8	...	Safari


```
% ps aux -r | head -4
```

ps	실행 중인 프로그램 목록을 보여주는 명령 (process status)
aux	모든 사용자(a), 보기 좋은 형식(u), 뒤에서 도는 것까지(x). 앞에 -를 붙이지 않는 옛 방식이다
-r	CPU를 많이 쓰는 순서로 정렬 (macOS)
 head -4	제목 줄과 상위 3개만

결과 읽기 %CPU가 298.4%인 node가 있다. 100%가 코어 하나를 꽉 채운 상태이니, 코어 세 개를 혼자 쓰고 있다는 뜻이다. PID(고유 번호)는 5123.

2 원인 그 프로그램이 무엇이고 언제부터 돌았는지 본다

이름만으로는 부족하다. 언제부터 실행 중인지, 어느 폴더에서 시작됐는지 확인하면 정체가 드러난다.

```
~ % ps -p 5123 -o pid,etime,command
```

```
PID      ELAPSED  COMMAND
```

```
5123    2-03:14:22 node /Users/jimin/old-project/server.js
```

```
% ps -p 5123 -o pid,etime,command
```

-p 5123	PID 5123번 프로그램만 본다 (process)
----------------	------------------------------

-o	보고 싶은 칸만 골라 출력한다 (output)
-----------	---------------------------

pid,etime,command	번호, 실행된 지 얼마나 됐는지(elapsed time), 실행 명령
--------------------------	--

결과 읽기 ELAPSED가 2-03:14:22, 즉 2일 3시간째 돌고 있다. 이를 전 old-project에서 켜 두고 잊어버린 개발 서버가 무한 반복에 빠진 것이다.

3 해결 정중하게 끄고, 안 되면 강제로 끈다

먼저 "정리하고 끝내 달라"는 정상 종료를 보낸다. 잠시 뒤에도 살아 있으면 그때 강제 종료한다.

```
~ % kill 5123
```

(출력 없음)

```
~ % ps -p 5123
```

```
  PID TTY          TIME CMD
 5123 ??        412:03.11 node /Users/jimin/old-
project/server.js
```

```
~ % kill -9 5123
```

(출력 없음)

명령어 해부

```
% kill 5123
```

kill

프로그램에 신호를 보내는 명령

5123

신호를 받을 PID. 숫자 옵션이 없으니 정상 종료 요청 (-15)이 간다

결과 읽기 조용하지만, 바로 이어서 확인해 보니 아직 살아 있다. 무한 반복에 빠진 프로그램은 부락을 들을 여유가 없기도 하다.

```
% kill -9 5123
```

-9

SIGKILL. 프로그램이 거부할 수 없는 강제 종료. 저장하지 않은 내용은 사라진다

결과 읽기 운영체제가 직접 끝낸다. 반드시 정상 종료를 먼저 시도하고, 안 될 때만 쓴다.

4 확인 다시 줄을 세워 본다

같은 명령으로 확인한다. 맨 위가 평범한 수치로 돌아왔다면 해결이다. 팬 소리도 곧 잦아든다.

```
~ % ps aux -r | head -3
USER    PID  %CPU  %MEM    ...   COMMAND
jimin   612   9.4   1.2    ...   WindowServer
jimin  4410   5.1   4.8    ...   Safari
```

다음부터는

개발 서버는 창을 닫기 전에 Control+C로 끄는 습관을 들이자. 실시간으로 지켜보고 싶을 때는 `top -o cpu`를 켜 두면 된다(q로 종료).

이 사례에서 쓴 명령어 · **ps** p.100 · **head** p.45 · **kill** p.104 · **top** p.102

CASE 3**외장하드가 "사용 중"이라며 추출되지 않는다****증상**

백업을 마치고 외장하드를 빼려는데 "'Backup' 디스크가 사용 중이므로 추출할 수 없습니다"라는 창이 뜬다. 열어 둔 창은 다 닫은 것 같은데.

1 진단

2 원인

3 해결

4 확인

1 진단 누가 그 디스크를 붙잡고 있는지 묻는다

"사용 중"이란 어떤 프로그램이 그 디스크 안의 파일이나 폴더를 열고 있다는 뜻이다. 열린 파일 목록에서 디스크 경로로 거른다.

```
~ % lsof +D /Volumes/Backup
```

COMMAND	PID	USER	FD	TYPE	NAME
---------	-----	------	----	------	------

Preview	7310	jimin	12r	REG	/Volumes/Backup/2026 결산.pdf
---------	------	-------	-----	-----	-----------------------------

zsh	7422	jimin	cwd	DIR	/Volumes/Backup
-----	------	-------	-----	-----	-----------------

```
% ls -lsof +D /Volumes/Backup
```

ls -lsof	어떤 프로그램이 무엇을 열고 있는지 보여주는 명령 (list open files)
+D	지정한 폴더와 그 아래 전체에서 열린 것을 찾는다 (Directory)
/Volumes/Backup	외장하드가 연결된 위치

결과 읽기 범인이 둘이다. 미리보기(Preview)가 PDF를 열고 있고, zsh 한 줄의 FD 칸에 cwd(current working directory)가 보인다. 다른 터미널 창이 cd로 그 디스크 안에 들어가 있는 것이다.

2 원인 의외의 범인은 나 자신

Finder 창은 다 닫았지만 미리보기로 결산 PDF를 열어 둔 채였고, 백업하던 터미널 창도 아직 /Volumes/Backup 안에 서 있었다. 폴더 안에 서 있는 것만으로도 "사용 중"이 된다.

3 해결 붙잡은 손을 하나씩 놓게 한다

미리보기는 앱을 종료하고, 디스크 안에 서 있던 터미널 창은 홈으로 나온다.

```
~ % killall Preview
(출력 없음)
Backup % cd ~
→ 프롬프트가 ~ % 로 바뀜 (그 창에서 실행)
```

명령어 해부

% killall Preview

killall	이름이 같은 프로그램을 모두 종료하는 명령. kill의 형제
Preview	미리보기 앱의 실제 프로그램 이름

결과 읽기 미리보기가 닫힌다. 저장할 게 있다면 앱에서 직접 닫는 편이 더 안전하다.

% cd ~

cd	폴더 이동
~	홈 폴더

결과 읽기 그 터미널 창이 디스크 밖으로 나왔다.

4 확인 아무도 붙잡고 있지 않은지 확인하고 추출한다

같은 lsof를 다시 실행해 아무것도 나오지 않으면 안전하다. 추출은 Finder에서 해도 되고 터미널에서 해도 된다.

```
~ % lsof +D /Volumes/Backup
(출력 없음)
~ % diskutil eject /Volumes/Backup
Disk /Volumes/Backup ejected
```

책 밖의 도구

diskutil eject — diskutil은 디스크를 관리하는 macOS 도구다. eject는 Finder의 추출 버튼과 같은 일을 한다.

다음부터는

외장하드 작업을 끝낸 터미널 창은 `cd ~` 로 빠져나오는 습관을 들이자. 추출이 안 될 때는 `ls -l +D` 하나면 이유를 알 수 있다.

이 사례에서 쓴 명령어 · **ls** p.107 · **kill** p.104 · **cd** p.16

CASE

4

스크립트를 실행하면 Permission denied가 뜬다

증상

동료가 준 배포 스크립트를 실행했더니 "permission denied"만 뜨고 아무 일도 일어나지 않는다. 설정 파일을 고치려 해도 같은 말이 나온다.

1 진단

2 원인

3 해결

1 진단 오류를 그대로 다시 보고, 권한을 확인한다

permission denied는 "허락되지 않았다"는 뜻이다. 누구에게 무엇이 허락되지 않았는지 ls -l로 확인한다.

```
~ % ./deploy.sh
zsh: permission denied: ./deploy.sh
~ % ls -l deploy.sh
-rw-r--r--  1 jimin  staff   420  9 30 10:12 deploy.sh
```

명령어 해부

% ./deploy.sh

./

"지금 폴더에 있는"이라는 뜻. 스크립트는 이렇게 경로를 붙여 실행한다

deploy.sh

실행하려는 스크립트 파일

결과 읽기 셸이 실행 자체를 거부했다.

```
% ls -l deploy.sh
```

-l

권한·주인·크기·날짜까지 자세히

결과 읽기 맨 앞 `-rw-r--r--`를 세 칸씩 끊어 읽는다. 주인(`rw-`)·그룹(`r--`)·그 외(`r--`). 어디에도 실행 권한 `x`가 없다. 파일을 메신저로 주고받으면서 실행 권한이 빠진 것이다.

3 해결 실행 권한을 더한다

파일에 `x`(실행) 권한을 붙여 주면 된다. 붙였는지 다시 확인하고 실행한다.

```
~ % chmod +x deploy.sh
```

(출력 없음)

```
~ % ls -l deploy.sh
```

```
-rwxr-xr-x 1 jimin staff 420  9 30 10:12 deploy.sh
```

```
~ % ./deploy.sh
```

배포를 시작합니다...

완료!

명령어 해부

```
% chmod +x deploy.sh
```

chmod

권한을 바꾸는 명령 (change mode)

+x

실행(`x`) 권한을 더한다(+). 누구에게인지 안 적으면 모두에게

결과 읽기 `rw-`가 `rwx`로 바뀌었다. 이제 실행된다.

2 원인 두 번째 증상: 이번엔 파일 주인이 다르다

이어서 설정 파일에 한 줄을 덧붙이려는데 또 거부당한다. 이번에는 권한이 아니라 주인이 문제다.

```
~ % echo "PORT=8080" >> config.env
zsh: permission denied: config.env
~ % ls -l config.env
-rw-r--r--  1 root  staff  88  9 29 17:40 config.env
```

명령어 해부

```
% ls -l config.env
```

root

세 번째 칸이 파일의 주인. root는 관리자 계정이다

결과 읽기 주인이 jimin이 아니라 root다. 어제 sudo를 붙여 이 파일을 만든 탓이다. 주인(root)만 쓸 수 있고(rw-), 나는 그 외(r--)라 읽기만 된다.

3 해결 주인을 나로 되돌린다

남의 파일 주인을 바꾸는 일이라 관리자 권한(sudo)이 필요하다. 한 번만 빌려서 주인을 나로 바꾼다.

```
~ % sudo chown $(whoami) config.env
Password:
~ % ls -l config.env
-rw-r--r--  1 jimin  staff  88  9 29 17:40 config.env
~ % echo "PORT=8080" >> config.env
(출력 없음)
```

```
% sudo chown $(whoami) config.env
```

sudo	이 명령 하나만 관리자 권한으로 실행. 맥 로그인 비밀번호호를 묻는다(입력해도 안 보이는 게 정상)
chown	파일 주인을 바꾸는 명령 (change owner)
\$(whoami)	whoami의 결과, 즉 내 사용자 이름(jimin)이 이 자리에 들어간다
config.env	주인을 바꿀 파일

결과 읽기 주인이 jimin으로 바뀌었고, 이제 sudo 없이도 자유롭게 고칠 수 있다.

다음부터는

sudo는 꼭 필요할 때만 쓰자. sudo로 만든 파일은 주인이 root가 되어 나중에 이런 문제를 만든다. Permission denied가 뜨면 고민하기 전에 `ls -l`부터.

이 사례에서 쓴 명령어 · [ls](#) p.19 · [chmod](#) p.71 · [echo](#) p.49 · [sudo](#) p.76 · [chown](#) p.74

CASE 5

분명 설치했는데 command not found가 뜬다

증상

Homebrew를 설치하고 안내대로 brew를 입력했더니 "command not found: brew". 설치가 실패한 걸까?

1 진단

2 원인

3 해결

4 확인

1 진단 프로그램이 정말 없는지, 못 찾는 것인지 가린다

셸이 모른다고 해서 파일이 없는 것은 아니다. 셸이 찾는 곳에 없을 뿐일 수 있다. 먼저 which로 셸의 입장을 듣고, ls로 실제 파일이 있는지 직접 본다.

```
~ % brew --version
zsh: command not found: brew
~ % which brew
brew not found
~ % ls /opt/homebrew/bin/brew
/opt/homebrew/bin/brew
```

명령어 해부

```
% which brew
```

which

명령어가 실제로 어디 있는 프로그램인지 찾는 명령

결과 읽기 셸이 brew를 찾지 못한다.

```
% ls /opt/homebrew/bin/brew
```

<code>/opt/homebrew/bin</code>	Apple 칩 맥에서 Homebrew가 프로그램을 설치하는 폴더
--------------------------------	-------------------------------------

결과 읽기 그런데 파일은 분명히 있다. 설치 성공했고, 셸이 이 폴더를 뒤져 보지 않는 것이 문제다.

2 원인 셸이 프로그램을 찾는 경로 목록을 본다

셸은 PATH라는 환경변수에 적힌 폴더들만 차례로 뒤져 명령을 찾는다. 그 목록을 출력해 본다.

```
~ % echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

명령어 해부

```
% echo $PATH
```

<code>echo</code>	받은 글자를 그대로 출력
<code>\$PATH</code>	PATH 변수의 값. \$를 붙이면 변수 이름 대신 그 안의 내용이 들어간다

결과 읽기 콜론(:)으로 구분된 폴더 다섯 곳 어디에도 /opt/homebrew/bin이 없다. 원인을 찾았다.

3 해결 PATH에 폴더를 추가하고, 설정 파일에도 적어 둔다

지금 창에 바로 추가해 되는지 확인한 뒤, 터미널을 새로 열어도 유지되도록 ~/.zshrc에 기록하고 적용한다.

```
~ % export PATH="/opt/homebrew/bin:$PATH"
(출력 없음)
~ % echo 'export PATH="/opt/homebrew/bin:$PATH"' >> ~/.zshrc
(출력 없음)
~ % source ~/.zshrc
(출력 없음)
```

명령어 해부

```
% export PATH="/opt/homebrew/bin:$PATH"
```

export	변수를 설정하고, 이 셸에서 실행하는 모든 프로그램에도 전달
/opt/homebrew/bin:	새로 추가할 폴더. 앞에 둘수록 먼저 찾는다
\$PATH	원래 목록을 그 뒤에 그대로 이어 붙인다. 이걸 빠뜨리면 기존 명령이 모두 사라진다

결과 읽기 지금 이 창에서는 brew가 된다. 하지만 창을 닫으면 사라진다.

```
% echo '...' >> ~/.zshrc
```

작은따옴표 '...'	안의 \$PATH가 지금 바로 풀리지 않고 글자 그대로 파일에 적히게 한다
>> ~/.zshrc	터미널이 켜질 때마다 읽는 설정 파일 끝에 덧붙인다

결과 읽기 이제 새 창에서도 자동으로 PATH가 설정된다. source로 지금 창에도 바로 반영했다.

4 확인 셸이 이제 찾는지 묻는다

처음 실패했던 명령을 그대로 다시 실행한다.

```
~ % which brew
/opt/homebrew/bin/brew
~ % brew --version
Homebrew 4.4.0
```

책 밖의 도구

brew — Homebrew는 맥용 프로그램 설치 도구다. Homebrew는 설치를 마칠 때 PATH 설정 명령을 화면에 안내해 주는데, 이 안내를 건너뛰면 지금 같은 일이 생긴다.

다음부터는

command not found를 만나면 which → ls → echo \$PATH 순서로 확인하자. "없는 것"과 "못 찾는 것"을 구별하면 대부분 5분 안에 해결된다.

이 사례에서 쓴 명령어 · **which** p.58 · **ls** p.19 · **echo** p.49 · **export** p.120 · **source** p.122

CASE 6**어제 받은 파일이 어디 있는지 모르겠다****증상**

어제 메일로 받은 강의 계약서를 어딘가에 저장했는데 파일 이름도, 폴더도 기억나지 않는다. 기억나는 건 "계약"이라는 단어가 들어 있다는 것뿐.

1 진단

2 원인

3 해결

1 진단 가장 흔한 곳부터 최근 순으로 본다

대부분은 다운로드 폴더에 있다. 최근에 생긴 파일부터 몇 개만 본다.

```
~ % ls -lt ~/Downloads | head -4
total 18244
-rw-r--r--  1 jimin  staff  2381022  9 30 09:02 IMG_2231.HEIC
-rw-r--r--  1 jimin  staff   120412  9 29 21:40 영수증.pdf
-rw-r--r--  1 jimin  staff   5501123  9 29 18:11 강의자료_v3.key
```

명령어 해부

% ls -lt ~/Downloads head -4	
-l	자세히
-t	최근에 수정된 순서로 (time)
 head -4	위에서 네 줄만. 첫 줄 total은 합계라 실제 파일은 세 개

결과 읽기 계약서처럼 보이는 파일은 없다. 다른 곳에 저장했나 보다.

2 원인 날짜와 내용으로 범위를 좁힌다

"어제 이후 생긴 PDF"로 찾아보고, 그래도 모르면 파일 이름이 아니라 내용으로 찾는다. Spotlight의 색인을 쓰면 문서 속 글자까지 순식간에 뒤진다.

```
% % find ~/Documents ~/Desktop -mtime -1 -name "*.pdf"
/Users/jimin/Desktop/스캔_0929.pdf
% % mdfind -onlyin ~ "위탁 계약"
/Users/jimin/Documents/업무/2026-수탁-최종.pdf
```

명령어 해부

```
% find ~/Documents ~/Desktop -mtime -1 -name "*.pdf"
```

-mtime -1	하루 안에 수정된 것 (modification time)
------------------	---------------------------------

-name "*.pdf"	이름이 .pdf로 끝나는 것
----------------------	-----------------

결과 읽기 스캔 파일 하나뿐이다. 이름에 계약이라는 단어가 없어 이름으로는 찾기 어렵다.

```
% mdfind -onlyin ~ "위탁 계약"
```

mdfind	Spotlight가 만들어 둔 색인에서 찾는 macOS 전용 명령
---------------	--------------------------------------

-onlyin ~	홈 폴더 안에서만
------------------	-----------

"위탁 계약"	파일 이름뿐 아니라 문서 안의 글자까지 찾는다
----------------	---------------------------

결과 읽기 파일 이름은 "2026-수탁-최종.pdf"였다. 이름에는 계약이 없었지만 본문에 있는 글자로 찾아냈다.

3 해결 Finder에서 바로 보여주고 연다

찾은 경로를 그대로 이용한다. 경로에 공백이나 한글이 있으면 따옴표로 감싼다.

```
~ % open -R "/Users/jimin/Documents/업무/2026-수탁-최종.pdf"
→ Finder에서 파일이 선택된 채로 폴더가 열림
~ % open "/Users/jimin/Documents/업무/2026-수탁-최종.pdf"
→ 미리보기로 계약서가 열림
```

명령어 해부

```
% open -R "..."
```

open

Finder에서 더블클릭한 것처럼 여는 macOS 전용 명령

-R

열지 않고 위치만 보여준다 (Reveal)

결과 읽기 파일이 어느 폴더에 있는지 눈으로 확인하고, 필요하면 원하는 곳으로 옮긴다.

다음부터는

파일 이름이 기억나지 않으면 find보다 mdfind가 빠르다. 내용 속 단어 하나만 기억해도 찾을 수 있다.

이 사례에서 쓴 명령어 · **ls** p.19 · **head** p.45 · **find** p.52 · **mdfind** p.134 · **open** p.129

CASE 7

인터넷이 안 된다. 어디가 문제일까

증상

브라우저에서 사이트가 열리지 않는다. 와이파이 아이콘은 멀쩡해 보인다.
공유기 문제일까, 인터넷 회사 문제일까, 내 맥 문제일까.

1 진단

2 원인

3 해결

4 확인

1 진단 가까운 곳부터 한 칸씩 멀리 신호를 보낸다

집 공유기 → 인터넷 너머의 서버(숫자 주소) → 이름 주소 순서로 확인한다. 어디서 끊기는지가 곧 원인의 위치다.

```
~ % ping -c 3 192.168.0.1
3 packets transmitted, 3 packets received, 0.0% packet loss
~ % ping -c 3 8.8.8.8
3 packets transmitted, 3 packets received, 0.0% packet loss
~ % ping -c 3 google.com
ping: cannot resolve google.com: Unknown host
```

명령어 해부

```
% ping -c 3 192.168.0.1
```

ping

상대에게 신호를 보내 응답이 오는지 확인하는 명령

-c 3

세 번만 보내고 멈춘다 (count)

192.168.0.1

흔히 쓰는 집 공유기 주소

결과 읽기 공유기까지는 잘 연결된다.

```
% ping -c 3 8.8.8.8
```

8.8.8.8

구글이 운영하는 공개 서버의 숫자 주소. 인터넷 너머까지 가는지 볼 때 쓴다

결과 읽기 인터넷 너머까지도 신호가 오간다. 인터넷 선 자체는 멀쩡하다.

```
% ping -c 3 google.com
```

google.com

이번엔 숫자가 아니라 이름 주소

결과 읽기 cannot resolve, 이름을 숫자 주소로 바꾸지 못했다. 숫자로는 되는데 이름으로는 안 된다.

2 원인 이름을 숫자로 바꿔 주는 DNS가 문제다

인터넷의 주소록 역할을 하는 DNS가 응답하지 않거나, 맥이 기억해 둔 주소록(캐시)이 꼬인 상태다. 연결은 멀쩡하니 공유기를 껐다 켜기 전에 맥의 주소록부터 비워본다.

3 해결 맥의 DNS 기억을 비운다

관리자 권한이 필요한 두 명령을 세미콜론(;)으로 이어 한 번에 실행한다.

```
~ % sudo dscacheutil -flushcache; sudo killall -HUP  
mDNSResponder  
Password:
```

```
% sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

sudo	관리자 권한으로 실행
dscacheutil -flushcache	맥이 기억해 둔 이름-주소 목록을 비운다 (macOS 도구)
;	앞 명령이 끝나면 이어서 다음 명령을 실행
killall -HUP	HUP 신호를 보낸다. 끄라는 뜻이 아니라 "설정을 다시 읽으라"는 뜻
mDNSResponder	macOS에서 DNS를 담당하는 프로그램

결과 읽기 비밀번호를 입력하면 조용히 끝난다. DNS 담당 프로그램이 주소록을 새로 채우기 시작한다.

4 확인 이름으로 다시 신호를 보내고, 사이트에 직접 물어본다

실패했던 명령을 다시 실행하고, 웹 서버가 정상 응답하는지까지 확인한다.

```
~ % ping -c 3 google.com
64 bytes from 142.250.206.46: icmp_seq=0 ttl=117 time=31.2 ms
3 packets transmitted, 3 packets received, 0.0% packet loss
~ % curl -I https://www.google.com
HTTP/2 200
content-type: text/html; charset=ISO-8859-1
```

명령어 해부

```
% curl -I https://www.google.com
```

curl	인터넷 주소와 데이터를 주고받는 명령
-------------	----------------------

-I	내용은 빼고 서버의 응답 정보만
-----------	-------------------

결과 읽기 HTTP/2 200은 "정상"이라는 뜻이다. 브라우저도 다시 열릴 것이다.

다음부터는

인터넷 문제는 가까운 곳부터 멀리 ping을 보내 보면 대부분 위치가 드러난다.
공유기 → 8.8.8.8 → 이름 주소, 이 세 줄을 기억해 두자.

이 사례에서 쓴 명령어 · **ping** p.85 · **sudo** p.76 · **kill** p.104 · **curl** p.87

CASE 8

문서 수십 개에 같은 오타가 있다

증상

강의 자료를 다 만들고 보니 "쿠버네티스"를 모든 문서에 "쿠버네티즈"로 잘못 썼다. 파일이 수십 개라 하나씩 열어 고치기엔 막막하다.

1 진단

3 해결

4 확인

1 진단 오타가 몇 개 파일에 퍼져 있는지 센다

고치기 전에 범위부터 안다. 오타가 들어 있는 파일 이름을 뽑아 개수를 세고, 실제로 어떻게 쓰였는지 몇 줄 본다.

```
~ % grep -rL "쿠버네티즈" docs/ | wc -l
37
~ % grep -rn "쿠버네티즈" docs/ | head -3
docs/03-intro.md:12:쿠버네티즈는 컨테이너를 관리하는 도구다.
docs/03-intro.md:40:쿠버네티즈 클러스터를 만든다.
docs/05-deploy.md:7:쿠버네티즈에 배포하기
```

명령어 해부

```
% grep -rL "쿠버네티즈" docs/ | wc -l
```

grep -r	docs 폴더 아래 모든 파일에서 찾는다
----------------	------------------------

-L	내용은 빼고 파일 이름만
-----------	---------------

 wc -l	파일 이름 줄 수 = 파일 개수
----------------	-------------------

결과 읽기 37개 파일에 오타가 있다. 손으로 고치면 한나절 일이다.


```
% grep -rn "쿠버네티즈" docs/ | head -3
```

-n 줄 번호를 붙인다

| head -3 세 줄만 맛보기

결과 읽기 단어 전체를 바꿔도 다른 말이 망가지지 않는지 실제 문장을 보고 판단한다.

3 해결 찾은 파일들에만 한꺼번에 바꾸기를 적용한다

grep이 찾은 파일 목록을 xargs로 sed에 넘긴다. 원본은 .bak 파일로 남겨 두어 문제가 생기면 되돌릴 수 있게 한다.

```
~ % grep -r\ "쿠버네티즈" docs/ | xargs sed -i .bak 's/쿠버네티즈/쿠버네티스/g'
(출력 없음)
```

```
% grep -r\l "쿠버네티즈" docs/ | xargs sed -i .bak 's/쿠버네티즈/쿠버네티스/g'
```

grep -r\l ... docs/

오타가 있는 파일 이름 목록을 만든다

| xargs

그 목록을 뒤 명령의 인자(재료)로 바꿔 붙인다. sed는 파이프를 받은 글자를 파일 이름으로 여기지 않기 때문에 필요하다

sed

글자를 바꾸는 명령 (stream editor)

-i .bak

원본 파일을 직접 고치되, 고치기 전 내용을 같은 이름.bak으로 남긴다

's/쿠버네티즈/쿠버네티스/g'

바꾸기(s) 규칙. 한 줄에 여러 번 있어도 모두(g)

결과 읽기 37개 파일이 한 번에 고쳐졌다. 각 파일 옆에 원래 내용이 담긴 .bak 파일이 생겼다.

4 확인 남은 오타가 없는지, 바뀐 곳이 맞는지 본다

같은 grep으로 오타가 0개인지 확인하고, 파일 하나를 골라 원본(.bak)과 비교해 의도한 곳만 바뀌었는지 눈으로 본다. 문제없으면 백업을 지운다.

```
~ % grep -r\l "쿠버네티즈" docs/ --exclude="*.bak" | wc -l
0
~ % diff docs/03-intro.md.bak docs/03-intro.md
12c12
< 쿠버네티즈는 컨테이너를 관리하는 도구다.
---
> 쿠버네티스는 컨테이너를 관리하는 도구다.
~ % find docs -name "*.bak" -delete
(출력 없음)
```

```
% grep -rl "쿠버네티즈" docs/ --exclude="*.bak" | wc -l
```

--exclude="*.bak" 백업 파일에는 옛 글자가 그대로 있으니 검사에서 뺀다

결과 읽기 0개. 오타가 모두 사라졌다.

```
% diff docs/03-intro.md.bak docs/03-intro.md
```

diff 두 파일을 비교

12c12 원본 12번째 줄이 새 파일 12번째 줄로 바뀌었다(c = change)

< 와 > <는 원본(왼쪽)에만 있던 줄, >는 새 파일(오른쪽)에만 있는 줄

결과 읽기 의도한 단어만 바뀌었다. 확인이 끝났으니 `find -delete`로 .bak 파일을 정리한다.

다음부터는

여러 파일을 한꺼번에 고칠 때는 `grep`으로 범위 확인 → `sed -i .bak`로 백업 남기며 수정 → `grep`과 `diff`로 확인 → 백업 삭제. 이 네 박자를 지키면 무서울 게 없다. 파일 이름에 공백이 있을 수 있다면 `grep -rl --null ... | xargs -0` 을 쓴다.

이 사례에서 쓴 명령어 · **grep** p.55 · **wc** p.61 · **head** p.45 · **xargs** p.68 · **sed** p.65 · **diff** p.110 · **find** p.52

맷으며

처음 터미널을 열었을 때 ls, cd, mv는 암호처럼 보였다. 이제 그 두 글자들로 폴더를 만들고, 막힌 곳의 원인을 찾고, 문제를 풀고, 결과를 확인할 수 있다. 단어는 50개뿐이지만 이어 붙이면 문장은 끝이 없다. 이제 당신도 두 글자로 말할 수 있다.

찾아보기

명령어 이름을 몰라도 괜찮다. 하고 싶은 일에서 출발하면 된다. 오른쪽 숫자는 페이지다.

이동과 파일

지금 어느 폴더에 있는지 모르겠다	pwd	14
다른 폴더로 이동하고 싶다	cd	16
폴더 안에 뭐가 있는지 보고 싶다	ls	19
숨김 파일까지 보고 싶다	ls	19
새 폴더를 만들고 싶다	mkdir	23
빈 파일을 만들고 싶다	touch	28
파일이나 폴더를 지우고 싶다	rm 30 rmdir	26
지워도 나중에 되살릴 수 있게 하고 싶다	rm	30
파일을 복사하거나 백업본을 만들고 싶다	cp 33 rsync	96
파일을 옮기거나 이름을 바꾸고 싶다	mv	36
바로가기(링크)를 만들고 싶다	ln	38

내용 보기와 가공

파일 내용을 바로 보고 싶다	cat	41
긴 파일을 천천히 넘겨 보고 싶다	less	43
파일 앞부분이나 끝부분만 보고 싶다	head 45 tail	47
로그를 실시간으로 지켜보고 싶다	tail	47
파일에 한 줄 써 넣고 싶다	echo	49
줄 수나 글자 수를 세고 싶다	wc	61
목록을 정렬하고 중복을 없애고 싶다	sort	63
파일 속 글자를 한꺼번에 바꾸고 싶다	sed	65
찾은 파일들에 한 번에 명령하고 싶다	xargs	68

찾기

파일이 어디 있는지 모르겠다	find 52	mdfind 134
파일 속 특정 글자를 찾고 싶다	grep 55	
설치한 프로그램이 어디 있는지 알고 싶다	which 58	
예전에 쓴 명령을 다시 찾고 싶다	history 124	grep 55
명령어 사용법을 모르겠다	man 126	

권한과 압축

Permission denied 오류가 난다	chmod 71	chown 74
관리자 권한이 필요하다고 나온다	sudo 76	
비밀번호를 쳐도 화면에 안 보인다	sudo 76	
SSH 키 권한 경고가 뜬다	chmod 71	ssh 92
여러 파일을 하나로 묶고 싶다	tar 79	zip 82
윈도우 사용자에게 압축해서 보내고 싶다	zip 82	
압축 파일을 풀고 싶다	tar 79	zip 82

인터넷과 서버

인터넷이 연결됐는지 확인하고 싶다	ping 85	
인터넷에서 파일을 받고 싶다	curl 87	wget 90
API를 테스트하고 싶다	curl 87	
서버에 접속하고 싶다	ssh 92	
서버와 파일을 주고받고 싶다	scp 94	rsync 96
외장하드에 백업하고 싶다	rsync 96	

맥의 상태

맥이 갑자기 느려졌다	top 102	ps 100
앱이 응답하지 않는다		kill 104
포트가 이미 사용 중이라고 나온다	lsof 107	kill 104
외장하드가 추출되지 않는다	lsof 107	
실행 중인 명령을 멈추고 싶다		Control+C 7
디스크 공간이 부족하다	df 115	du 113 find 52
두 파일이 어디가 다른지 보고 싶다		diff 110

터미널 환경과 맥 전용

자주 쓰는 긴 명령을 줄이고 싶다	alias 118
설치했는데 command not found가 뜬다	which 58 export 120
설정을 바꿨는데 적용이 안 된다	source 122
터미널에서 Finder로 넘어가고 싶다	open 129
명령 결과를 클립보드에 복사하고 싶다	pbcopy 132
작업 중에 맥이 잠들지 않게 하고 싶다	caffeinate 136
작업이 끝나면 소리로 알림을 받고 싶다	say 138

두 글자로 말하는 사람들 — 맥 터미널, 50개의 단어로 컴퓨터와 대화하는 법 · zsh 기본 프롬프트 (%) 기준으로 작성 · 예시의 파일·폴더명은 이해를 돕기 위한 가상의 예시이다.

